dg_xch_utils — DIVERGENCES ledger

CLVM VM / block-generator divergences

Correctness ledger for the hand-rolled CLVM VM (`core/src/clvm/`) and block generator (`core/src/consensus/block_generator.rs`), surfaced by porting chia-blockchain's CLVM/generator test corpus.

Each divergence below is asserted by a test that encodes **Chia's canonical behavior** and is marked `#[ignore = "DIVERGENCE-<n>: ..."]` so `cargo test` stays green. Removing the `#[ignore]` reproduces the failure. Tests that are *not* ignored are proven-correct behavior and are locked in.

The original harvest changed no production code; subsequent fix phases (each a separate `full-node` commit, cited in the relevant section) have closed every *characterized* divergence.

Status summary: DIVERGENCE-1, -2, -3, -4, -5, -6, -7, -8, -10, -11, -12, -13, -14, -15, -16, -17, -18, -19 are **FIXED**; DIVERGENCE-9 is an architectural absence subsumed by the block store (no bug). DIVERGENCE-24 (CLVM BLS operators: invalid-point and cost-timing semantics vs clvmr) is **OPEN — characterized, `#[ignore]`d** in `core/tests/bls_ops_differential.rs`. The DIV-13 tail is CLOSED: the live epoch boundary at mainnet 9,146,880 was crossed 2026-08-14 with zero rejections (ses-carrying records + per-height epoch schedule + epoch-depth backfill, proven against the real retarget). The soft-fork-9 flag set (`SIMPLE_GENERATOR | CANONICAL_INTS | LIMIT_SPENDS`, the DIV-17 companion) is now **characterized and ported** — see the "Soft fork 9 flag set" section below. The genesis long-sync campaign (DIV-19 onward) hunts the historical-regime walls. DIVERGENCE-11 is the additions/removals `merkle_set` root: flipping the DIV-10 end-to-end test to the **real `FoliageTransactionBlock`** surfaced `BadAdditionRoot` because dg_xch's merkle set did not match chia's radix-tree node scheme (`EMPTY/TERMINAL/MIDDLE` tags, `hashdown` prefix, terminal-pair collapse) and its addition leaves dropped single-coin `hash_coin_ids`. With DIVERGENCE-11 fixed, the end-to-end validator runs the COMPLETE `validate_body` (execution + all roots + agg-sig + cost + fees) to green on real mainnet blocks with real foliage — the offline proof-of-sync. DIVERGENCE-3 — the mainnet height-9138874 block-generator stall (`InvalidBlockSolution`) — was fixed (a rejected zero-argument REMARK condition, plus an under-counted `coinid` operator). DIVERGENCE-10 is the stage that surfaced *after* the DIV-3 fix on the live node: the same batch advanced past cost/conditions and failed the generator-HASH check (`InvalidTransactionsGeneratorHash`) because `transactions_generator_root` tree-hashed the decompressed generator instead of hashing its raw serialized bytes — now fixed, with the real-block root coverage the DIV-3 harvest lacked.

DIVERGENCE-1 — arithmetic operators decode atoms as unsigned, not signed — FIXED

Status: FIXED on `full-node` (signed two's-complement atom decode commit). `impl From<&AtomBuf<'_>> for SExpNumber` in `core/src/clvm/sexp_ext.rs` now sign-extends any atom up to 16 bytes into an `i128` (high bit of the first byte ⇒ negative) and defers longer atoms to the already-signed `number_from_slice` (`BigInt::from_signed_bytes_be`). The int→atom encode path was already canonical minimal signed two's-complement (`impl_ints!` for `i128` and `formatting::bigint_to_bytes` in `core/src/clvm/sexp.rs`), so only the decode changed.

Reference (canonical atom↔int contract): - Decode: chia `chia/util/casts.py::int_from_bytes` — empty ⇒ `0`, else `int.from_bytes(blob, "big", signed=True)`. Identical in the CLVM VM authority (`clvm/casts.py::int_from_bytes`). - Encode: chia `chia/util/casts.py::int_to_bytes` — `0` ⇒ `b""`, otherwise the minimal signed encoding trimming leading `0x00`/`0xff` (`while len(r) > 1 and r[0] == (0xFF if r[1] & 0x80 else 0): r = r[1:]`). dg_xch's `impl_ints!` trim loop mirrors this exactly.

Was (suspected code path): `core/src/clvm/sexp_ext.rs`, `impl From<&AtomBuf<'_>> for SExpNumber`. Atom bytes were decoded as **unsigned** big-endian (`buf[0] as i128`; `u16/u32/u64/u128::from_be_bytes(..) as i128`; and a zero-padded `i128::from_be_bytes` for the remaining lengths). CLVM integers are **signed two's-complement** big-endian. This decode is reached via `two_ints()` / `<(SExpNumber, usize)>::try_from` in `core/src/clvm/more_ops.rs` for `op_add` (16), `op_subtract` (17), `op_multiply` (18), `op_div` (19) and `op_divmod` (20). Operators that instead use `number_from_slice` (`op_gr` 21, `op_gr_bytes` 10, `op_ash` 22, …) already decoded signed and were unaffected.

Chia oracle: `chia/_tests/clvm/test_program.py::test_run` — `div.run_with_flags(100000, 0, [10, -5])` → atom `0xfe` (–2), cost `1107`.

Concrete inputs — expected (Chia) vs actual (dg_xch):

input	expected	actual	why
`(/ 10 -5)`	`-2` (`0xfe`)	`0` (`()`)	`-5` = `0xfb` decoded as `251` ⇒ `10/251 = 0`
`(+ -1 1)`	`0`	`256` (`0x0100`)	`-1` = `0xff` decoded as `255` ⇒ `255+1 = 256`
`(/ 2 5)` on `[10, -5]`	result `0xfe`, cost `1107`	result `()`, cost differs	same unsigned decode of `-5`

Evidence tests (now live + passing): - `clvm::more_ops::tests::div_negative_divisor_floors_to_minus_two` - `clvm::more_ops::tests::add_with_negative_operand_is_signed` -

`clvm::program::tests::run_div_negative_operand_result_and_cost` (`(/ 2 5)` on `[10, -5]` ⇒ atom `0xfe`, cost `1107`)

Added signed-boundary coverage: - `clvm::more_ops::tests::signed_boundary_decode_and_roundtrip` (`0x80`⇒−128, `0x0080`⇒+128, `-1`↔`0xff`, `0`↔empty atom, `-1 * -1 == 1`) - `clvm::more_ops::tests::div_negative_dividend_floors_toward_neg_infinity` (`(/ -7 2) == -4`, atom `0xfc`) - `clvm::more_ops::tests::greater_than_with_negative_operand_is_signed` (`(> -1 1) == ()`, `(> 1 -1) == 1`)

Consensus relevance: any puzzle/generator arithmetic on a high-bit (negative) operand previously computed the wrong value. This was a *candidate* contributing cause for DIVERGENCE-3; measuring the blast radius (see below) ruled it out as the cause of the height-9138874 stall.

DIVERGENCE-2 — `Program::uncurry` errors on a non-curried program — FIXED

Status: FIXED on `full-node` (CLVM/API layer). Harvested from `chia/_tests/clvm/test_program.py::test_uncurry_not_curried` (and the `test_uncurry_top_level_garbage` / `test_uncurry_not_pair` / `test_uncurry_args_garbage` family that share the contract).

Chia's `uncurry` never raises: any program not in curried apply/quote form returns the pair `(self, nil)` rather than an error. dg_xch's `core/src/clvm/program.rs::uncurry` previously used `inner_match`, which returned `Err(InvalidInput("expected: 02"))` when the top node was not the apply/curry shape — and, worse, its `ref_list()`-flatten approach both (a) would index-panic on a program with fewer than three top-level elements and (b) **failed to reject over-long lists** (top-level or args garbage), a false-positive-uncurry risk.

Reference (canonical `uncurry` contract): `chia/types/blockchain_format/program.py:183` `Program.uncurry` (delegated to by the free `uncurry` at `:298`). It matches `(2 (1 . <mod>) <args>)` inside a `try`, and its `except ValueError / TypeError / EvalError` fallback is **`return self, self.to(0)`**. The precise second element is `self.to(0)` — chia's `int_to_bytes(0)` is the empty atom `b""`, i.e. **nil** (`SExp::from(0) == NULL_SEXP` in dg_xch). Curried-vs-not detection is the exact apply/quote structural shape:

- top level must be a **proper 3-element list** (`ev, quoted_inner, args_list = self.as_iter()` — a 4th element or a short list makes the unpack raise), with `ev == 0x02`;
- `quoted_inner` must be a **pair** whose first is `0x01` (its rest is `mod`);
- each args cons must be a proper 3-list `(4 (1 . <arg>) <rest>)` with head `0x04` and a `(1 . <arg>)` quote pair; and
- the args chain must terminate in the atom `0x01`.

Any deviation ⇒ `(self, nil)`.

Return-shape note: the second element is the **nil atom** `()` (from chia's `self.to(0)`), not an empty *list* wrapper — `args.sexp().nullp()` holds. The successful-uncurry path is unchanged (`curry(mod, [a, b]).uncurry() == (mod, [a, b])` still holds).

The fix: `uncurry` was rewritten to walk the apply/quote shape via `maybe_pair`/`first`/`rest` (a `three()` inner helper enforces the exact "proper 3-list" unpack, so both top-level and args garbage are rejected) and to return `Ok((self, nil))` on **any** mismatch instead of `Err`. The `Result` signature is retained so existing callers keep compiling; the only behavioral change is `Err → Ok((self, nil))` for non-curried input. Call sites were audited (`puzzles/src/utils.rs`, `puzzles/src/clvm_puzzles.rs`): all either guard on a `tree_hash` comparison, an `as_list().len()` check, or an `is_*_inner_puzzle` predicate that is unaffected by the fallback, or become *more* chia-faithful (`get_inner_puzzle_from_puzzle` now returns `Ok(None)` for a non-curried puzzle, matching chia's non-raising `uncurry`); none panic and none change a block-validation result.

Concrete input: `uncurry` of `assemble("(+ 2 5)")` — expected `(Program("(+ 2 5)"), nil)`; former actual `Err(InvalidInput("expected: 02"))`.

Evidence test (now live + passing): - `clvm::program::tests::uncurry_not_curried_returns_program_and_nil` (was `#[ignore]`d) — `(+ 2 5)` uncurries to `(itself, nil)`.

Added contract coverage (mirrors the chia oracle's whole `test_uncurry_*` family): - `uncurry_top_level_garbage_returns_program_and_nil` — a 4th top-level element. - `uncurry_not_pair_returns_program_and_nil` — the module slot is a bare atom. - `uncurry_args_garbage_returns_program_and_nil` — an over-long args cons (the false-positive-uncurry guard). - `uncurry_plain_atom_and_plain_pair_return_program_and_nil` — a plain atom, nil, and a proper 3-list whose head is not `0x02` all return `(self, nil)` without panicking. - Pre-existing `uncurry_positive_case` and `curry_then_uncurry_round_trips` remain green — the regression guard that the successful path is untouched.

Consensus relevance: API-level only. `uncurry` is a wallet/analysis helper, not on the block-validation path. Low risk, but it had broken the documented contract (and the false-positive over-long-list case was a latent correctness bug for any analysis code that trusted the returned module/args).

DIVERGENCE-3 — mainnet block generator at height 9138874 rejected (`InvalidBlockSolution`) — FIXED

Status: FIXED on `full-node`. Two independent defects on the generator-output→conditions path, found by threading the swallowed error through and then measuring the residual cost against real mainnet blocks. The live node stalled with `InvalidBlockSolution` on the 9138828..9138859 batch; both defects are exercised by that batch.

Surfacing the real error (STEP 1)

`execute_block_generator_result` mapped every condition-extraction failure through `|_| ChiaError::InvalidBlockSolution`, hiding the cause. Threading the underlying `ClvmError` through (now permanent — see *Instrumentation* below) and running the (then-`#[ignore]`d) test surfaced, **verbatim**:

```
DIV3-SURFACE: condition extraction failed at height 9138874
(simple_generator=true):
IoError(Custom { kind: InvalidData, error: "Invalid Condition Remark
No Vars: - (q . ())" })
```

i.e. a **REMARK condition with zero arguments** — the CLVM `(1)` / `(q . ())` — rejected by dg_xch's condition parser.

Defect A — zero-argument REMARK (and unknown opcodes) wrongly rejected

`op_code_with_args_from_sexp` (`core/src/blockchain/condition_with_args.rs`) had a blanket guard `if vars.is_empty() && opcode != AssertEphemeral { Err(...) }`, and `from_opcode_with_args` enforced `Remark` args `!= 1`. chia's condition parser treats **REMARK (opcode 1) as `Ok(Condition::Skip)` — always true, ignoring ALL of its arguments regardless of count (including zero)**, and treats an **unknown opcode as a no-op** in consensus (non-strict) mode.

Reference: chia_rs `crates/chia-consensus/src/conditions.rs`: - `parse_args`, final arm: `REMARK => { /* always true, we always ignore arguments */ Ok(Condition::Skip) }`. - `parse_conditions`: `let Some(op) = parse_opcode(a, first(a, c)?, flags) else { if NO_UNKNOWN_CONDS { return Err(InvalidConditionOpcode) } ... continue; }` — unknown opcodes are ignored (still cost-charged under COST_CONDITIONS) rather than rejected. - Extra trailing arguments are likewise ignored for every condition unless the mempool-only `STRICT_ARGS_COUNT` flag is set (`maybe_check_args_terminator`).

Fix: REMARK maps to `ConditionWithArgs::Remark(Message::new(Vec::new()))` for any argument count (the message is never read downstream), and the empty-vars guard now also exempts `Remark` and `Unknown`. Every other opcode still requires its leading argument(s) via the per-opcode checks in `from_opcode_with_args`, so a genuinely-truncated condition is still rejected.

Defect B — `coinid` operator under-counted by 320 (missing malloc cost)

With REMARK parsing fixed, the run advanced and failed on a **cost mismatch**. The exact-cost test asserts `conds.cost == transactions_info.cost`. Measuring the per-block residual against the fixture's ten self-contained transaction blocks: the flat condition-cost model (`CREATE_COIN_COST` = 1,800,000 per CREATE_COIN, `AGG_SIG_COST` = 1,200,000 per AGG_SIG_*, nothing else) reproduces chia's cost **exactly on 7 of 10 blocks**, and is short by exactly **320 / 320 / 640** on the other three — the blocks (and only those blocks) that invoke the CLVM `coinid` operator (opcode 48) **1 / 1 / 2** times. Residual = `320 × coinid-count`.

`op_coinid` (`core/src/clvm/more_ops.rs`) returned the bare `COIN_ID_COST` (480) and built the result atom by hand, **omitting the malloc cost of the freshly allocated 32-byte coin id** — `32 × MALLOC_COST_PER_BYTE (10) = 320` — that every other hashing op charges (e.g. `op_sha256` returns via `new_atom_and_cost`).

Reference: clvmr `src/more_ops.rs::op_coinid` returns `new_atom_and_cost(a, cost, &ret)` where `cost = COINID_COST` (the same `SHA256_BASE + 3·PER_ARG + 72·PER_BYTE − 153 = 480` formula dg_xch uses), and `new_atom_and_cost` adds `len × MALLOC_COST_PER_BYTE`; for the 32-byte coin id that is `480 + 320 = 800`.

Fix: `op_coinid` now returns `new_atom_and_cost(COIN_ID_COST, coin_id.as_ref())`, adding the 320 malloc cost and matching clvmr. After this, the flat model plus the corrected `coinid` cost reproduces `transactions_info.cost` **exactly on all ten blocks**.

Cost-model note (why the flat model is correct here)

chia's *newer* cost model (`ClvmFlags::NEW_COST_MODEL` / `ConsensusFlags::COST_CONDITIONS`: per-spend `SPEND_COST`, the reduced `NEW_CREATE_COIN_COST`, per-message / per-generic condition costs, and the `NEW_*` operator costs) is a **later hard fork that is not yet active at height ~9.13M**, even though the v2 "simple" generator already is. This was confirmed empirically: applying that model over-charged the exact-cost blocks (e.g. +6,336,200 at 9138874), whereas the flat pre-fork model matches to the byte. dg_xch therefore keeps the flat condition-cost model unchanged for these heights; the only cost bug was the `coinid` malloc omission.

Instrumentation (STEP 3.3 — the swallow must not return)

`execute_block_generator_result` no longer discards the cause. `map_condition_error` (`core/src/consensus/block_generator.rs`) maps recognizable `ClvmErrors` to their faithful `ChiaError` (`InvalidPublicKey → InvalidCondition`, `CostExceeded → BlockCostExceedsMax`, `DoubleSpend → DoubleSpend`, `DuplicateCreate → DuplicateOutput`) and logs the underlying error at **error** level; a structurally malformed generator output still falls through to `InvalidBlockSolution`.

Evidence tests (now live + passing) in `core/tests/respond_blocks_generator_execution.rs`: - `real_mainnet_generator_executes_to_block_cost` (was `#[ignore]`d) — every self-contained transaction-block generator in the fixture (heights 9138874, 9138876, 9138879, 9138880, 9138883, 9138887, 9138891, 9138895, 9138900, 9138903) decompresses, runs, and executes to its own `transactions_info.cost` exactly. Requires `checked >= 2`, so the fix is proven on many real blocks including the 9138828..9138859-adjacent batch that stalled the live node. - `corrupted_generator_is_still_rejected` — a structurally valid simple generator (`(q . ())`) whose output is nil rather than a spend list is still rejected as `InvalidBlockSolution` (and `execute_block_generator` surfaces error code 2), so the instrumentation refinement does not admit a bad solution.

Consensus relevance: blocking for mainnet sync — this is the exact stall. A standard transaction block emitting a zero-argument REMARK (or spending a coin whose puzzle uses the `coinid` operator) could not be validated before this fix.

DIVERGENCE-4 — negative / out-of-range height/seconds condition arguments — FIXED

Status: FIXED on `full-node`. **Batch 2** (conditions layer). Harvested from `chia/_tests/core/full_node/test_conditions.py::TestConditions::test_condition`.

CLVM condition integers are signed two's-complement, but chia stores time-lock bounds in fixed-width `u32`/`u64` slots. dg_xch's condition-argument decode in `core/src/blockchain/condition_with_args.rs` (`from_opcode_with_args`) previously diverged two ways:

- **HEIGHT_*** (`u32`) decoded via `formatting::u32_from_slice`, which is **unsigned** (`u32_from_slice_impl(buf, /*signed=*/false)`): `-1` (atom `0xff`) became `255`, converting a no-op into a real future constraint. Same signed-vs-unsigned root cause as DIVERGENCE-1, in the condition-arg path.
- **SECONDS_*** (`u64`) decoded via `number_from_slice` (signed) → `u64_from_bigint`, which **rejected** a negative outright (`"cannot convert negative integer to u64"`), so the generator errored before validation — over-rejecting a spend chia accepts.

The fix: decode the arg as a signed `BigInt` (`number_from_slice`) and **saturate** it into the slot with the two new inherent helpers in `core/src/formatting.rs` — `saturating_u32_from_bigint` / `saturating_u64_from_bigint` (negative ⇒ `0`, `> type::MAX` ⇒ `type::MAX`). Applied to all eight time-lock opcodes; `ASSERT_MY_BIRTH_HEIGHT/SECONDS` (equality checks, opcodes 74/75) are deliberately left on the old decode. No change to `block_generator.rs`: the assertion **direction** already encoded in the validator (`>=`/`max`-fold for `ASSERT_*`; `<`/`min`-fold for `ASSERT_BEFORE_*`) resolves which saturated end is a no-op and which fails.

Reference (canonical per-opcode rule): `chia/_tests/core/full_node/test_conditions.py::TestConditions::test_condition` parametrized rows (chain: 4 prior blocks, spend in the 5th; prev-tx-block height 3 / ts 10030; spent coin born height 2 / ts 10020; genesis ts 10000, 10 s/block). The two families move in **opposite** directions for an impossible bound:

opcode (dg_xch enum value)	negative arg
`ASSERT_SECONDS_RELATIVE` (80)	no-op, **PASS**
`ASSERT_SECONDS_ABSOLUTE` (81)	no-op, **PASS**
`ASSERT_HEIGHT_RELATIVE` (82)	no-op, **PASS**
`ASSERT_HEIGHT_ABSOLUTE` (83)	no-op, **PASS**
`ASSERT_BEFORE_SECONDS_RELATIVE` (84)	**FAIL** (`ASSERT_BEFORE_SECONDS_RELATIVE_FAILED`
`ASSERT_BEFORE_SECONDS_ABSOLUTE` (85)	**FAIL** (`ASSERT_BEFORE_SECONDS_ABSOLUTE_FAILED`

opcode (dg_xch enum value)	negative arg
ASSERT_BEFORE_HEIGHT_RELATIVE (86)	**FAIL** (ASSERT_BEFORE_HEIGHT_RELATIVE_FAILED)
ASSERT_BEFORE_HEIGHT_ABSOLUTE (87)	**FAIL** (ASSERT_BEFORE_HEIGHT_ABSOLUTE_FAILED)

Chia additionally distinguishes ASSERT_BEFORE_*_ABSOLUTE 0 as IMPOSSIBLE_*_ABSOLUTE_CONSTRAINTS (an aggregate lower-bound-vs-upper-bound check). dg_xch has no separate impossible-constraint code and collapses every BEFORE_* failure into AssertHeightAbsoluteFailed / AssertSecondsAbsoluteFailed (absolute) or AssertHeight/SecondsRelativeFailed (relative). The **accept/reject direction** is identical to chia; only the specific error code is coarser.

> Note: an earlier draft of this ledger listed (85 -1) as ASSERT_BEFORE_HEIGHT_ABSOLUTE. Verified against core/src/blockchain/condition_opcode.rs: 85 is ASSERT_BEFORE_SECONDS_ABSOLUTE; ASSERT_BEFORE_HEIGHT_ABSOLUTE is 87.

Concrete inputs — expected (Chia) vs former actual (dg_xch):

condition	expected
(83 -1) ASSERT_HEIGHT_ABSOLUTE, spent at height 0	pass (no-op)
(81 -1) ASSERT_SECONDS_ABSOLUTE	pass (no-op)
(87 -1) ASSERT_BEFORE_HEIGHT_ABSOLUTE	ASSERT_BEFORE_HEIGHT_ABSOLUTE_FAI

Evidence tests (now live + passing) in core/tests/block_generator.rs: - negative_height_absolute_is_noop_like_chia - negative_seconds_absolute_is_noop_like_chia

Added both-direction coverage (the under-rejection was the dangerous one): - negative_before_height_absolute_rejects_like_chia - negative_before_seconds_absolute_rejects_like_chia - negative_before_height_relative_rejects_like_chia - negative_before_seconds_relative_rejects_like_chia - overflow_height_absolute_rejects_like_chia (assert-past, out-of-range ⇒ FAIL) - overflow_seconds_absolute_rejects_like_chia - overflow_before_height_absolute_is_noop_like_chia (before, out-of-range ⇒ PASS) - overflow_before_seconds_absolute_is_noop_like_chia

Consensus relevance: high. Any real spend carrying a negative or out-of-range height/seconds assertion is now accepted/rejected consistently with the network. The BEFORE_* negative under-rejection (a spend the network rejects being admitted) was the dangerous direction and is the one this fix closes.

DIVERGENCE-5 — block generator ref-list resolution is not wired — FIXED

Status: FIXED on `full-node` (node + stores layers). Harvested from `chia/_tests/blockchain/test_get_block_generator.py`.

Chia's `get_block_generator(lookup, block)` assembles a block-level generator by looking up each referenced prior-block generator **by height** through a storage callback (`lookup_block_generators` → `BlockStore.get_generators_at`, the `in_main_chain=1` query) and supplying them **in ref-list order** (`chia/full_node/full_node.py:2500-2505`: `generator_args = [generators[height] for height in block.transactions_generator_ref_list]`). A referenced height with no confirmed generator is a hard failure (`get_generators_at`: `if len(generators) != len(heights): raise KeyError(Err.GENERATOR_REF_HAS_NO_GENERATOR)`; `get_generator`: `gen is None ⇒ Err.GENERATOR_REF_HAS_NO_GENERATOR`).

dg_xch previously **failed closed**: `node/src/engine.rs::validate_body` returned `Invalid("generator ref list resolution not wired in this phase")` on any non-empty `transactions_generator_ref_list`, hardcoded `generator_refs: Vec::new()`, and computed `refs_root` from `&[]` — so no real ref-list block could validate.

The fix (node + stores; the `core` executor already consumed resolved refs): - **Store lookup added** — `BlockStore::get_generator_at_height(h) -> Option<SerializedProgram>` (`stores/src/traits.rs`, impl in `stores/src/sqlite/block.rs`, forwarded in `stores/src/arc.rs`). A single confirmed-main-chain join (`block_body JOIN block_record ... WHERE height = ? AND in_main_chain = 1`) decompresses the cold body and returns its generator — the mirror of chia `get_generators_at`. Returns `None` on a miss (absent height or no generator). - **Engine resolution** — `Engine::resolve_generator_refs(ref_list)` fetches each height's generator in ref-list order, mapping a miss to `ChiaError::GeneratorRefHasNoGenerator` (matches chia). Called from `add_block_with_delta` (the async, store-touching step) and threaded through the sync `derive_delta` into `validate_body`. - **refs_root** is now computed from the **actual** referenced heights (`transactions_generator_refs_root(&block.transactions_generator_ref_list)`) and validated against `ti.generator_refs_root`; a disagreeing root is rejected.

Reference: `chia/consensus/get_block_generator.py::get_block_generator`, `chia/consensus/blockchain.py::lookup_block_generators`, `chia/full_node/block_store.py::get_generators_at` / `get_generator`, `chia/full_node/full_node.py:2500-2505`.

Evidence tests (now live + passing): - `core/tests/get_block_generator.rs::ref_list_block_resolves_prior_generator_from_storage` (was `#[ignore]`d) — block 4,671,894 with its height-4,671,893 reference resolved executes and yields conditions; the empty-ref path cannot reproduce them. - `core/tests/get_block_generator.rs::generator_refs_root_is_order_sensitive`, `generator_refs_root_of_real_heights_differs_from_empty` — the refs_root rejection predicate `validate_body` relies on.

Added storage-resolution coverage (the actual gap) in `node/tests/generator_ref_resolution.rs`: - `get_generator_at_height_returns_the_confirmed_generator` — by-height resolution from the store. - `get_generator_at_height_missing_is_none` — an absent height and a confirmed-but-bodiless height both miss. - `resolve_generator_refs_preserves_ref_list_order` — **(a)** two refs, order follows the ref-list (and reverses when the list reverses); refs_root is order-sensitive and differs from the empty root. - `resolve_generator_refs_missing_height_is_validation_failure` — **(b)** an absent or partially-missing ref fails closed with `GeneratorRefHasNoGenerator`, never a silent pass.

Consensus relevance: blocking for mainnet sync — a transaction block that references a prior block's generator (block-level CLVM back-reference compression) can now be validated. dg_xch resolves from the confirmed main chain (`in_main_chain=1`), the equivalent of chia's `get_generators_at`; chia's extra fork/reorg lookup path (`lookup_fork_chain`) is an optimization for references that fall on a not-yet-confirmed fork and is out of scope for this fix.

DIVERGENCE-6 (FIXED) — BLS G1 infinity public key not rejected in AGG_SIG conditions

Batch 2 (conditions layer). Harvested from `chia/_tests/core/full_node/test_conditions.py::TestConditions::test_agg_sig_infinity`.

Chia rejects the BLS12-381 G1 identity/infinity element (`0xc0` followed by 47 zero bytes) as an `AGG_SIG_*` public key with `Err.INVALID_CONDITION` (`chia/util/errors.py:39` → `INVALID_CONDITION = 10`). dg_xch's condition path parsed it as an ordinary `Bytes48`: `from_opcode_with_args` did no infinity check, and `verify_agg_sig_unsafe_message` (`core/src/blockchain/utils.rs`) only screens the forbidden message *suffix*, not the key. The infinity key was accepted into the `SpendBundleConditions`, deferring any failure to signature verification.

Chia oracle: `test_agg_sig_infinity` (`chia/_tests/core/full_node/test_conditions.py:481-494`) — every `AGG_SIG_*` opcode with the infinity pubkey ⇒ `Err.INVALID_CONDITION`.

Covered opcode family (all eight, per the oracle's `@pytest.mark.parametrize` at `test_conditions.py:470-479`): `AGG_SIG_UNSAFE` (49), `AGG_SIG_ME` (50), and the soft-fork-5 additions `AGG_SIG_PARENT` (43), `AGG_SIG_PUZZLE` (44), `AGG_SIG_AMOUNT` (45), `AGG_SIG_PUZZLE_AMOUNT` (46), `AGG_SIG_PARENT_AMOUNT` (47), `AGG_SIG_PARENT_PUZZLE` (48). All eight are defined in `core/src/blockchain/condition_opcode.rs` and carried by `ConditionWithArgs`.

Activation: the rule is a soft-fork-5 rule that has long since activated on mainnet. The reference no longer even carries a `SOFT_FORK5_HEIGHT` constant (`chia/consensus/default_constants.py` now only defines `SOFT_FORK8/9`), and the oracle asserts `Err.INVALID_CONDITION` **unconditionally** at mainnet heights. dg_xch's `ConsensusConstants` (`core/src/consensus/constants.rs`) likewise carries no

`soft_fork5_height` field, so there is no per-height gate to hook: the rejection is implemented as always-on, matching the oracle. (This is a fresh-genesis fork reusing the mainnet condition ruleset, well past every soft-fork-5 boundary.)

Fix: `ConditionWithArgs::agg_sig_infinity_pubkey` (`core/src/blockchain/condition_with_args.rs`) matches the eight `AGG_SIG_*` variants and returns the key iff it is the canonical G1 infinity encoding. `spend_from_conditions` (`core/src/consensus/block_generator.rs`) checks it at the top of the condition loop — during aggregation, **before** `validate_block_aggregate_signature` — and returns `ClvmError::InvalidPublicKey`, which `execute_block_generator_result` maps to `ChiaError::InvalidCondition` (Err code 10). Non-infinity keys are untouched (no change to the deferred signature check).

Evidence test (now live, passing): `core/tests/block_generator.rs::agg_sig_infinity_pubkey_is_rejected_like_chia` (asserts `Err(ChiaError::InvalidCondition)`, code 10). Added coverage: `agg_sig_infinity_pubkey_rejected_for_every_agg_sig_opcode` (all eight opcodes) and `agg_sig_non_infinity_pubkey_is_still_accepted` (ordinary key and a `0xc0`-prefixed near-miss both accepted — no over-rejection).

Consensus relevance: medium. An infinity pubkey has no corresponding secret key, so it is only exploitable in constructions that expect the key to be rejected; still a genuine deviation from the network's condition-validity rules.

Ruled out (looked like a divergence, was a test-transcription error)

- `chia/_tests/clvm/test_chialisp_deserialization.py::test_deserialization_large_numbers` first failed with `BadEncoding` in the native parser. The failure was a hand-transcription slip in the fixture hex (a mis-split multi-line string literal), **not** a parser divergence. With the exact source bytes the native `sexp_from_bytes` round-trips it correctly (`clvm::parser::tests::deserialization_large_numbers_round_trips` is live/green).

DIVERGENCE-7 — `Option` presence tag `> 1` accepted instead of rejected — FIXED

Status: FIXED on `full-node`. **Batch 3** (streamable/wire layer). Harvested from `chia/_tests/core/util/test_streamable.py::test_parse_optional`.

Chia's `parse_optional` requires the 1-byte presence tag to be **exactly** `0x00` (None) or `0x01` (Some); any other value raises `ValueError`. dg_xch's `impl<T> ChiaSerialize for Option<T>` (`serialize/src/lib.rs`, `from_bytes`) tested `bool_buf[0] > 0` and treated **every** non-zero byte as Some, then decoded the inner value. This is the same "a non-canonical flag byte is not rejected" leniency family as DIVERGENCE-1 (unsigned/loose decode).

Reference (canonical optional tag contract):
`chia/util/streamable.py::parse_optional` — reads one byte; `bytes([0])` ⇒ `None`, `bytes([1])` ⇒ `parse_inner_type_f(f)`, `else: raise ValueError("Optional must be 0 or 1")`.

The fix: `impl<T> ChiaSerialize for Option<T>::from_bytes` now matches the tag byte: `0 ⇒ Ok(None)`, `1 ⇒ Ok(Some(inner))`, and any `other ⇒ Err(InvalidInput, "Optional must be 0 or 1, found: {other}")`. The `0`/`1` paths are byte-for-byte unchanged; only the previously-lenient `2..=255` range now rejects.

Chia oracle: `test_parse_optional` — `parse_optional(b"\x02\x00", parse_bool)` and `parse_optional(b"\xff\x00", parse_bool)` both raise `ValueError`.

Concrete inputs — expected (Chia) vs former actual (dg_xch):

input (as `Option<bool>`)	expected	former actual
`\x02\x00`	`Err` (tag must be 0/1)	`Ok(Some(false))`
`\xff\x00`	`Err`	`Ok(Some(false))`

Evidence test (now live + passing): - `core/tests/streamable_wire.rs::option_presence_tag_must_be_zero_or_one_like_chia` — tags `0x02` and `0xff` reject; `0x00 ⇒ None`, `0x01 <inner> ⇒ Some` still decode.

Consensus relevance: low–medium. Honest serializers only ever emit `0x00`/`0x01`, so a block produced by a conforming node round-trips identically. The gap is malformed-input rejection: a peer message (or crafted bytes) with a `2..=255` presence tag is accepted by dg_xch and rejected by chia, a decoder-hardening / DoS-surface deviation rather than a divergence on honestly-encoded blocks.

DIVERGENCE-8 — trailing bytes after a value are not rejected — FIXED

Status: FIXED on `full-node`. **Batch 3** (streamable/wire layer). Harvested from `chia/_tests/core/util/test_streamable.py::test_from_bytes_rejects_trailing_bytes_rust_types`.

Chia's top-level `from_bytes(blob)` requires the blob to be **fully consumed** — `Coin.from_bytes(bytes(coin) + b"\x00")` raises `ValueError`. dg_xch's `ChiaSerialize::from_bytes(&mut Cursor)` is cursor-based and reads only the bytes the type needs, leaving any trailing bytes un-inspected in the cursor and returning `Ok`. There was no "whole buffer consumed" check at the `ChiaSerialize` boundary.

Reference (the from_bytes/parse split — the crux): `chia/util/streamable.py`. Chia has **two distinct entry points**, and only the outer one checks consumption: - `Streamable.from_bytes(cls, blob)` — the top-level entry: `f = io.BytesIO(blob); parsed = cls.parse(f); remainder = f.read(); if remainder != b"": raise ValueError(f"{cls.__name__}:`

{len(remainder)} bytes not consumed"). - Streamable.parse(cls, f) — the per-field reader: iterates streamable_fields() calling each field's parse_function(f) off the shared stream and does **not** check for a remainder. Nested composite fields legitimately leave bytes on the stream for the **next** field, so the consumption check must live **only** at the outer from_bytes, never in parse. Putting it inside per-field parsing would break every nested/tuple/list/Option decode.

The fix (mirrors that split without renaming the trait method): the existing cursor-based ChiaSerialize::from_bytes(&mut Cursor) is dg_xch's parse — it stays untouched and continues to leave trailing bytes for the next field (nested decoding depends on this). A new **provided** trait method ChiaSerialize::from_bytes_full(bytes: &[u8], version) -> Result<Self, Error> (serialize/src/lib.rs) is the outer entry: it wraps bytes in a Cursor, calls Self::from_bytes, then rejects if cursor.position() != bytes.len() with InvalidData, "{n} bytes not consumed" — the exact analog of chia's from_bytes→parse boundary. No signature change, no new dep, no free helper; the nested cursor reader and the derive macro are unchanged.

Chia oracle: test_from_bytes_rejects_trailing_bytes_rust_types — Coin.from_bytes(valid + b"\x00") raises; likewise SpendBundle.

Concrete input: Coin::from_bytes_full over to_bytes(coin) ++ [0x00] (73 bytes). Expected: rejected (trailing byte); an exactly-72-byte buffer decodes.

Evidence test (now live + passing): - core/tests/streamable_wire.rs::from_bytes_rejects_trailing_bytes_like_chia — the outer from_bytes_full rejects the padded 73-byte buffer and accepts the exact 72-byte one, while the nested cursor from_bytes still stops at position 72 (proving the split).

Added split-guard coverage: - from_bytes_full_rejects_trailing_bytes_for_primitive_and_composite — the outer entry rejects a trailing byte for both a primitive (u32) and a nested composite ((Vec<Vec<u32>>, (Option<u32>, Option<u32>), (u32, String, Vec<u8>))), and accepts the exact buffers. - nested_optional_leaves_bytes_for_next_field — a nested Option (both None and Some) hands the following u32 to the next tuple field instead of rejecting it as "trailing", so the consumption check is proven to be outer-only.

The pre-existing composite round-trip (nested_composite_round_trips_byte_for_byte) and the 121 KB clvm_generator.bin round-trip remain green — the regression guard that the consumption check did not over-tighten nested decoding.

Consensus relevance: low–medium. Same malformed-input class as DIVERGENCE-7: honestly-serialized messages have no trailing bytes, but a padded/oversized frame that chia rejects is now rejectable at the type-decode boundary via the outer from_bytes_full entry (the message-framing layer calls it for whole-buffer decodes).

DIVERGENCE-9 — no standalone block-height map component (subsumed by the block store)

Batch 3 (stores layer). Harvested from `chia/_tests/core/full_node/test_block_height_map.py`.

Chia maintains a dedicated `BlockHeightMap` (a persisted, contiguity-enforcing, auto-extending height→hash structure with its own `ensure_capacity` / `update_height` / rollback surface and its own unsupported-version guard). dg_xch has **no equivalent component**: the height→hash contract is served entirely by `BlockStore::get_block_record_by_height` over the `in_main_chain` confirmation pointers, and rollback/contiguity fall out of `set_peak`'s pointer flip. This is an architectural absence, not a runtime bug — the height→hash lookups chia's map provides are all reproducible through the block store.

Chia oracle: `test_empty_chain`, `test_peak_only_chain`, `test_update_height_auto_extends` (contiguity + peak boundary).

Coverage (live, not ignored): - `stores/tests/block_store.rs::height_map_contiguity_via_block_store` proves the block store serves the height→hash contract (every in-range height resolves to the canonical hash after `set_peak`; nothing resolves above the peak).

Consensus relevance: none for correctness; documented so the absence is a recorded design decision rather than a silent gap. If a future fix phase needs the map's O(1) contiguous-array reads (weight-proof / long-range sync hot paths), it would add the component; today the block-store query is the single source of truth.

Hardening note (not a numbered divergence) — length-prefixed pre-allocation is unbounded

`String::from_bytes` (`serialize/src/lib.rs`) reads a u32 length prefix and immediately does `vec![0u8; vec_len]` **before** reading the payload, so a crafted `\xff\xff\xff\xff` prefix requests a ~4 GiB allocation up front. Chia's `parse_str`/`parse_bytes` validate the declared length against the available bytes first. The *observable result matches* (both ultimately return an error on a short buffer — dg_xch via the subsequent `read_exact` failure), so this is not a correctness divergence and carries no ignored test (a faithful reproduction would risk OOM-ing the runner). It is recorded as a decode-hardening item: the fix is to bound/So-cross-check the declared length against remaining input before allocating. The `Vec<T>::from_bytes` count path does **not** pre-allocate and errors cleanly on the first missing element (proven live by `core/tests/streamable_wire.rs::oversized_list_count_rejects_without_panicking`).

DIVERGENCE-10 — `transactions_generator_root` tree-hashes the generator instead of hashing its raw bytes — FIXED

Status: FIXED on `full-node`. Surfaced by the **live mainnet node** one validation stage *after* the DIVERGENCE-3 fix: the 9138828..9138859 batch no longer failed with `InvalidBlockSolution` (cost/conditions now validate), it advanced to the generator-HASH check and failed there —

```
sync node error: consensus rejected block:
InvalidTransactionsGeneratorHash
```

`ChiaError::InvalidTransactionsGeneratorHash` is returned from two places in `node/src/engine.rs::validate_body` — the **gen_root** check (`transactions_generator_root(...) != ti.generator_root`) and the **refs_root** check (`transactions_generator_refs_root(...) != ti.generator_refs_root`). The DIVERGENCE-3 test only called `execute_block_generator_result` (execution→cost) and never computed either root, so this path was **untested** against real mainnet blocks.

STEP 1 — which root, and by how much (offline, on the real fixture)

A throwaway reproduction (`core/tests/div10_repro.rs`) computed both roots for every transaction block in `core/tests/fixtures/respond_blocks_mainnet_9138873_9138904.bin` and compared them to the block's own `transactions_info` roots. **Verbatim:**

```
DIV10 GEN_ROOT MISMATCH height=9138874 backref=true raw_len=12223
  current(tree_hash) =
cdd70c7dd0c26b6a0cd3091bc4e7ae15cf39256d644907cc98646b273fe2954c
  correct(sha256)    =
dbcada6f50b67197961f4f35d2d0780c6ea18dd5a92f5b63c56058b343cd9a6b
  declared           =
dbcada6f50b67197961f4f35d2d0780c6ea18dd5a92f5b63c56058b343cd9a6b
DIV10 GEN_ROOT MISMATCH height=9138876 backref=true raw_len=2588
  current(tree_hash) =
78c506f5b3b8fb84a636544e194bd8f6e3fd2e0d6e5db9d2b3f962c13feede9d
  correct(sha256)    =
138ee6bddf89b116a07b728b43422ba241177aa4beaf1a8d033d6b2affc6eecb
  declared           =
138ee6bddf89b116a07b728b43422ba241177aa4beaf1a8d033d6b2affc6eecb
DIV10 SUMMARY checked=10 gen_mismatch=10 refs_mismatch=0
```

So it is the **gen_root** that mismatches — on **all 10** self-contained transaction blocks (all back-referenced generators) — while **refs_root matches on all 10**. Decisively: `sha256(bytes(generator))` **equals the declared root byte-for-byte**, and the tree hash of the decompressed generator does not.

Root cause

`core/src/consensus/block_generator.rs::transactions_generator_root` had a height gate:

```
if height >= constants.hard_fork_height {
    Ok(generator.to_program_backrefs()?.tree_hash())   // WRONG
} else {
    Ok(Bytes32::new(hash_256(generator.as_ref())))      // correct
}
```

At mainnet `hard_fork_height = 5_496_000` (`core/src/consensus/constants.rs`), every block at ~9.13M takes the first branch and computes the `sha256tree` (`Program::get_tree_hash`) of the **decompressed** generator. Chia never does this. The DIVERGENCE-1 raw-bytes `SerializedProgram` change proved the wire bytes round-trip byte-exact, but the *hash is taken over those raw bytes*, not over the decompressed tree — so the tree-hash branch was the regression surface DIV-1's note flagged.

Reference (chia computes `std_hash` of the raw generator bytes, always)

Identical on the producer and validator sides, with **no height gate and no tree hash**:

- **Validation:** `chia/consensus/block_body_validation.py` (rule 7a) — `if std_hash(bytes(block.transactions_generator)) != block.transactions_info.generator_root: return Err.INVALID_TRANSACTIONS_GENERATOR_HASH`. Mirrored in `chia/consensus/blockchain.py` (the `receive_block` pre-validation).
- **Production:** `chia/consensus/block_creation.py::create_foliage` — `generator_hash = std_hash(new_block_gen.program)` (where `new_block_gen.program` is a `SerializedProgram`, i.e. the raw on-wire generator bytes, back-references and all).

`bytes(SerializedProgram)` is the verbatim wire encoding; dg_xch's `SerializedProgram::as_ref()` is exactly those bytes (DIVERGENCE-1). Hence chia's `std_hash(bytes(generator))` ≡ dg_xch's `hash_256(generator.as_ref())` — the former `else` branch was already correct for **all** heights.

For completeness the **refs_root** was also grounded and confirmed correct (it did not mismatch): `chia/consensus/block_body_validation.py` (rule 8a) — `generator_refs_hash = std_hash(b"".join([i.stream_to_bytes() for i in block.transactions_generator_ref_list]))`, and `bytes([1] * 32)` when the ref list is empty; `chia/consensus/block_creation.py` computes it the same way. `uint32.stream_to_bytes()` is 4 big-endian bytes, which dg_xch's `u32::to_bytes` (`impl_primitives!` → `to_be_bytes`) matches; `transactions_generator_refs_root` already returns `[1; 32]` for the empty list.

The fix

`transactions_generator_root` now unconditionally returns `Bytes32::new(hash_256(generator.as_ref()))` — the raw-bytes sha256, no height branch, no `to_program_backrefs`, no `tree_hash`. The height gate, the `ConsensusConstants` parameter, and the fallible `Result` (sha256 cannot fail, matching

chia's infallible `std_hash`) are removed; the two call sites (`validate_transaction_block` in the same file, and `Engine::validate_body` in `node/src/engine.rs`) are updated to the `(&SerializedProgram) -> Bytes32` signature.

Evidence tests (now live + passing) in `core/tests/respond_blocks_generator_root.rs`

- `real_mainnet_generator_roots_match_declared` — for every transaction block in the fixture, `transactions_generator_root` and `transactions_generator_refs_root` equal the block's declared `generator_root` / `generator_refs_root`. Asserts `checked >= 2` and `back_referenced >= 1` (the exact untested surface). This is the root coverage the DIVERGENCE-3 harvest lacked; it is **red** on the pre-fix tree-hash implementation (the STEP-1 mismatch above) and green after the fix.
- `real_mainnet_transaction_block_validates_end_to_end` — runs the real body validator `validate_transaction_block` (execution + gen_root + refs_root + aggregate-signature verify + cost + fees + additions/removals roots + `transactions_info_hash`) on real mainnet blocks and requires it to **accept** `>= 2` of them, proving the whole path the live node runs, not isolated pieces.
- `tampered_generator_is_still_rejected` — flipping one generator byte changes its sha256 root, so `validate_transaction_block` still rejects it (the check is not loosened): a genuinely wrong generator does not validate.

Consensus relevance: blocking for mainnet sync — this is the exact live stall after DIV-3. No post-hard-fork transaction block (i.e. every mainnet transaction block today, all of which use back-referenced generators) could pass the generator-identity check before this fix.

DIVERGENCE-11 — additions/removals `merkle_set` root does not match chia's node scheme (`BadAdditionRoot`) — FIXED

Status: FIXED on `full-node`. Surfaced by flipping the DIVERGENCE-10 end-to-end test (`core/tests/respond_blocks_generator_root.rs::real_mainnet_transaction_block_validates_end_to_end`) to attach the **real `FoliageTransactionBlock`** (`Some`) instead of `None`. With real foliage, `validate_transaction_block` runs the two foliage-root comparisons — `additions_root` and `removals_root` — that `None` skipped. On the fixture's real mainnet transaction blocks (heights 9138873–9138904) execution, gen_root, refs_root, agg-sig, cost, and fees all pass, then validation rejects with `ChiaError::BadAdditionRoot`.

This is the exact integration gap the DIVERGENCE-3/-10 unit harvest missed: those tests never supplied foliage, so the additions/removals merkle-set roots were never computed against a real block. The end-to-end test with real foliage is now the oracle — it runs precisely the checks the live node runs to sync a transaction block.

Root cause — two independent bugs, both in `core/src/consensus/block_generator.rs`

(a) The merkle-set primitive `merkle_set_root` was not chia's merkle set. chia's merkle set is a binary radix tree over the big-endian bits of the (already-hashed) 32-byte leaves, with node-type tags `EMPTY=0 / TERMINAL=1 / MIDDLE=2` and a fixed `hashdown(buf) = sha256(bytes([0] * 30) + buf)`. The prior implementation instead used `sha256(bytes([1]) + key)` for a leaf **at every position** (not just at the root), `sha256(bytes([2]) + left + right)` for an interior node (wrong prefix: no 30-byte zero pad, no child type tags), `sha256(b"")` for the empty set (chia uses all-zero `bytes32`), and — most consequentially — it **collapsed** any level where all leaves shared a bit (`if split == 0 || split == len { recurse(bit+1) }`), which chia does **not** do for a genuine `Middle` subtree. Every one of these diverges from chia:

- empty root: chia `bytes32.zeros`, not `sha256(b"")` — `chia/_tests/core/test_merkle_set.py::test_merkle_set_0`.
- single-leaf root: chia `sha256(b"\1" + key)` — `test_merkle_set_1`.
- interior node: chia `hashdown(ltag ++ rtag ++ lhash ++ rhash)` where a **terminal child contributes its raw key** (not `sha256(\1+key)`) — `test_merkle_set_2` (`hashdown(b"\1\1" + b + a)`).
- shared-bit levels: chia emits an **empty-sibling** middle for a `Middle` subtree (`test_merkle_set_5`: `hashdown(b"\0\2" + BLANK + sub)` / `hashdown(b"\2\0" + sub + BLANK)`), but path-compresses a **terminal pair** (`MiddleDbl`) straight through (`test_merkle_set_3`: `{b,c}` becomes `hashdown(b"\1\1" + b + c)` with no empty chain). The distinguishing rule — recurse the non-empty side, wrap only if the child is a `Middle`, forward a `Terminal`/`MiddleDbl` up unchanged — is chia_rs `crates/chia-consensus/src/merkle_tree.rs::generate_merkle_tree_recurse`.

(b) `canonical_additions_root` built the wrong leaves. chia's `validate_block_merkle_roots` (`chia/consensus/block_body_validation.py`, mirrored in `chia/consensus/block_creation.py`) groups created-coin ids by puzzle hash and appends **both** the puzzle hash **and** `hash_coin_ids(coin_ids)` as leaves for **every** group — including single-coin groups. The prior code appended the coin-id hash **only when `coin_ids.len() > 1`**, dropping the leaf for every single-coin puzzle hash (the common case), and it computed that hash by sorting the coin ids **ascending** and concatenating. chia's `hash_coin_ids` (`chia/types/blockchain_format/coin.py`) is: one id ⇒ `std_hash(id)` (no prefix); many ⇒ `sort(reverse=True)` (**descending**), concatenate, `std_hash`.

The fix

- `merkle_set_root` rewritten as chia's radix tree: `MerkleNodeType {Terminal, Middle, MiddleDbl}` with `tag()` (`Terminal→1`, `Middle`/`MiddleDbl→2`), `merkle_hashdown(ltag, rtag, l, r) = sha256([0;30] ++ ltag ++ rtag ++ l ++ r)`, empty set ⇒ all-zero root, single leaf finalized as `sha256([1] ++ key)`, interior terminals contribute their raw key, and the recurse/collapse rule above (`merkle_set_recurse`). `bit_is_set` is unchanged (bit-clear ⇒ left).
- `canonical_additions_root` now emits `puzzle_hash` **and** `hash_coin_ids(coin_ids)` for every group unconditionally; new `hash_coin_ids` mirrors chia exactly (single ⇒ `sha256(id)`; many ⇒ descending sort + concat + `sha256`).

- `canonical_removals_root` is unchanged: it already feeds spent-coin ids (`spend.coin_id`, i.e. chia's `coin.name()` for each removal) into `merkle_set_root`, which now matches chia — `chia/consensus/block_body_validation.py` (`removals_root = compute_merkle_set_root(tx_removals)`).

Evidence tests (live + passing) in `core/tests/respond_blocks_generator_root.rs`

- `real_mainnet_transaction_block_validates_end_to_end` — now attaches the **real foliage** (`Some`) and asserts `validate_transaction_block` accepts `>= 2` blocks **with foliage**, additionally checking the returned `additions_root`/`removals_root` equal the block's declared foliage roots. This is the sync-ready oracle.
- `real_mainnet_additions_and_removals_roots_match_foliage` — isolates the two roots: on every real block with foliage, computed `additions_root` and `removals_root` equal the declared foliage roots (`>= 2` blocks).
- `tampered_foliage_roots_are_rejected` — corrupting the declared `additions_root` ⇒ `BadAdditionRoot`; corrupting `removals_root` ⇒ `BadRemovalRoot`. Guards the checks against loosening.
- `block_generator::merkle_set_tests` (unit) — pins `merkle_set_root` against chia's published vectors `test_merkle_set_0..5` (including the empty-child chain of `test_merkle_set_5` and the terminal-pair collapse of `test_merkle_set_3`) and `hash_coin_ids` against `chia/types/blockchain_format/coin.py`.

Consensus relevance: blocking for mainnet sync — the additions/removals roots are foliage-committed and checked on **every** transaction block. Before this fix, no real transaction block with a `CREATE_COIN` output (i.e. effectively all of them) could pass `validate_body`. With the fix, the end-to-end validator runs the COMPLETE `validate_body` path (execution + gen_root + refs_root + agg-sig + cost + fees + `transactions_info_hash` + additions_root + removals_root) to green on the fixture's real blocks **with real foliage** — the offline proof-of-sync.

DIVERGENCE-12 — a fabricated `required_iters == 0` poisons the difficulty retarget (`Required iters 0 is not below the sp interval iters`) — FIXED

Status: FIXED on `full-node`. Surfaced by the live mainnet node the moment DIVERGENCE-10 (`generator_root`) and DIVERGENCE-11 (additions/removals `merkle_set` roots) cleared the block-body wall: the peak advanced past 9138828 and then stalled one block later, in a **different subsystem** — the header / proof-of-space `required_iters` path, not `validate_body`:

```
WARN follow step failed, retrying next tick  from=9138830 to=9138861
  error="sync node error: io error: Required iters 0 is not below the
sp interval iters 2097152, 134217728 or not > 0."
```

Surfaced computation — plumbing, not formula

The verbatim message is emitted by `calculate_ip_iters` (`core/src/consensus/pot_iterations.rs`) and ONLY when `required_iters == 0` (`required_iters >= sp_interval_iters || required_iters == 0`) — mirroring chia `chia/consensus/pot_iterations.py::calculate_ip_iters`. The sp/ip bounds `2097152 / 134217728` are `sub_slot_iters / num_sps` and `sub_slot_iters`.

A `required_iters` of exactly 0 can never come from the FORMULA: `calculate_iterations_quality` (`core/src/consensus/pot_iterations.rs`, chia `pot_iterations.py::calculate_iterations_quality`) clamps its result with `max(1, …)`, so a valid proof of space yields at least 1. The weight-proof path (phases 1–6 green on this node) already computes `required_iters` correctly for real blocks, confirming the formula is right. **The 0 was a fabricated/stored value fed back into the check.**

The unwrapped call site that emits the verbatim string is `BlockRecord::sp_total_iters -> sp_sub_slot_total_iters -> ip_sub_slot_total_iters -> BlockRecord::ip_iters -> calculate_ip_iters` (`core/src/blockchain/block_record.rs`). It is reached from `get_next_sub_slot_iters_and_difficulty` (`core/src/consensus/difficulty_adjustment.rs`), which reads `prev_b.sp_total_iters(constants)?` — a faithful port of chia `chia/consensus/difficulty_adjustment.py::get_next_sub_slot_iters_and_difficulty` (`sp_total_iters = prev_b.sp_total_iters(constants)`). The read is correct; the **input** was not. The two header-validation call sites of `calculate_ip_iters` wrap the error (`map_err(|_| e("ip_iters"))`), so the *verbatim* message can only escape through this difficulty-retarget read — which localizes the failure to the block-sync/follow path, exactly where the live log points.

Root cause — `Engine::derive_required_iters` fabricated a 0 (`node/src/engine.rs`)

On the follow/`add_block` path, `Engine::compute_record` calls `Engine::derive_required_iters` to stamp each new record's `required_iters`. When the deep ancestor context needed for full PoW/VDF header validation was **not yet cached** (a checkpoint/bootstrap entry point — the first blocks after fast-sync hands off to follow), it returned `Ok(0)` and stored that 0 in the record. Its own comment even asserted "required_iters is not read by fork choice or coin state" — true, but it IS read by the **difficulty retarget** of every later block through `prev_b.sp_total_iters()`. The stall lands at checkpoint+2: checkpoint+1 is confirmed with a fabricated `required_iters == 0` (tolerated, nothing reads it yet), and checkpoint+2 is the first block whose full validation calls `get_next_sub_slot_iters_and_difficulty(prev_b = checkpoint+1)`, reads its `sp_total_iters`, and hits `calculate_ip_iters(…, 0)`.

chia never stores a fabricated `required_iters`: every `BlockRecord` carries its true value (computed via `calculate_iterations_quality` during header validation), so chia's identical `get_next_sub_slot_iters_and_difficulty` read is always safe. dg_xch's fabricated 0 is the divergence.

Computed vs expected for the fixture's real blocks (`recent_chain_mainnet_9054524_9054620`, the same region the T030 header test uses, sub-slot-iters `574619648`, sp interval `8978432`):

height	fabricated (old fallback)	chia formula / weight-proof reference
9054611	0 (poison)	2155695
9054612	0 (poison)	7701062

The fix

`Engine::derive_required_iters` no longer fabricates a 0 when the deep ancestor context is missing. It calls the new `Engine::pospace_required_iters`, which computes the true value from the proof of space alone — no VDF/ancestor chain required:

- pospace challenge = the block's own declared `pos_ss_cc_challenge_hash` (exactly what chia's `validate_unfinished_header_block` feeds to `validate_pospace_and_get_required_iters` after its `INVALID_CC_CHALLENGE` check confirms `get_block_challenge(...) == pos_ss_cc_challenge_hash`);
- `cc_sp_hash` from `challenge_chain_sp_vdf.output.hash()` (challenge when absent);
- difficulty = the block's weight increment over `prev` (chia enforces `block.weight == prev.weight + difficulty`, the `INVALID_WEIGHT` check, so this equals the epoch difficulty the full path derives via `get_next_sub_slot_iters_and_difficulty`);
- `prev_transaction_block_height = 0` — unused by dg_xch's `validate_pospace_and_get_required_iters` (its pospace quality is height-based only).

These are the identical inputs the full-validation path uses for a valid block, so the stored `required_iters` is the same whether the deep context is present or not — and it is always in `(0, sp_interval_iters)`, never the fabricated 0 that crashes the retarget. When the context IS cached, the full PoW/VDF validator still runs (unchanged) and returns the same value.

This was PLUMBING, not the iters formula. `calculate_iterations_quality` / `calculate_ip_iters` were already correct (weight-proof-proven); the bug was feeding a fabricated zero into a correct check.

Evidence tests (live + passing) in `node/tests/required_iters_real_blocks.rs`

- `pospace_required_iters_matches_full_validation_and_is_in_range` — for the fixture's real mainnet blocks, the ancestor-independent pospace computation (exactly what the fix now stores) lands in `(0, sp_interval_iters)`, equals the full PoW/VDF validator's `required_iters`, and equals the weight-proof reference `GOLDEN` values. Also pins that the fix's weight-delta difficulty equals the epoch difficulty chia's `INVALID_WEIGHT` check enforces. This is the header/pospace coverage the block-body harvest (DIV-3/-10/-11) never had.
- `stored_zero_required_iters_poisons_the_difficulty_retarget` — the guard: a real prev record whose `required_iters` is blanked to 0 makes `get_next_sub_slot_iters_and_difficulty` fail with the exact live verbatim

message, while the real value retargets cleanly. Proves a genuinely-degenerate `required_iters == 0` stays fatal — the fix is "never store 0", not "tolerate 0".

Consensus relevance: blocking for mainnet sync — this is the exact live stall after DIV-10/-11. Every block whose full validation runs the difficulty retarget (i.e. every block past the fast-sync→follow handoff) reads its predecessor's `required_iters` through `sp_total_iters`; a single fabricated 0 in that window halts the sync.

DIVERGENCE-13 — the anchor body-confirm fabricates `sub_slot_iters_starting` into the block record, poisoning the sp-interval bound (`Required iters N is not below the sp interval iters`) — FIXED

Status: FIXED on `full-node` (anchor-confirm candidate-ssi fix commit).

dg_xch path: `node/src/engine.rs` `compute_record_from_header` (the record's `sub_slot_iters`), fed by `Engine::add_block_with_delta`.

chia oracle: `chia/consensus/blockchain.py` `add_block` — every `block_to_block_record` is fed `(sub_slot_iters, difficulty)` from `get_next_sub_slot_iters_and_difficulty`; records are epoch-adjusted at creation, never blind-inherited and never defaulted. At a weight-proof checkpoint, where the deep ancestry that walk needs is absent, chia substitutes the proof's own summaries (`chia/consensus/multiprocess_validation.py` `pre_validate_blocks_multiprocessing`, `wp_summaries`).

The live failure (both DIV-13 sightings, one cause)

The old `compute_record_from_header` set the record's ssi as `prev.map_or(sub_slot_iters_starting, |p| p.sub_slot_iters)`. At the fast-sync anchor the previous record does not exist, so the FIRST confirmed record was stamped with the genesis constant `134,217,728` — overwriting the headers-first candidate record that carried the weight-proof-attested epoch value (`570,425,344` at the mainnet 9,143,053 anchor). Every descendant then inherited the poison. `derive_required_iters`'s full validation reads that ssi back through `get_next_sub_slot_iters_and_difficulty` (between-boundary pass-through returns `prev_b.sub_slot_iters`), so the `calculate_ip_iters` bound became `134,217,728 / 64 = 2,097,152` instead of the true `570,425,344 / 64 = 8,912,896`.

A block is only rejected when its true `required_iters` lands BETWEEN the two bounds — which is why the wall was sporadic and always near sub-epoch boundaries by coincidence of plot quality, not boundary logic:

height	required_iters	poisoned bound	true bound	verdict (old)
9,141,129	2,405,336	2,097,152	8,978,432	rejected (wrong)
9,143,058	5,786,293	2,097,152	8,912,896	rejected (wrong)

Ground truth from the live store (`chain.db`, anchor 9,143,053): headers-first candidate records carried ssi `570,425,344`; the body-confirmed records for the same heights carried `134,217,728`. The confirm path was destroying correct data.

The fix

`add_block_with_delta` now pre-fetches the stored candidate record for the block being confirmed and threads `candidate_ssi` down to `compute_record_from_header`, whose ssi resolution mirrors chia:

1. ancestry walkable (prev + prev.prev cached) → `get_next_sub_slot_iters_and_difficulty` (chia-exact, epoch-adjusted — the same call full validation makes);
2. otherwise → the headers-first candidate record's attested ssi (chia's `wp_summaries` substitution, carried by the record the header pass stored);
3. otherwise → inherit `prev.sub_slot_iters` (correct between boundaries);
4. genesis only → `sub_slot_iters_starting`.

Fabricating the genesis constant mid-chain — the DIV-12 disease in a second organ — is no longer possible on any path that has better information.

Evidence test (live) in `node/tests/t056_fast_sync_advances_from_empty.rs`

- `anchor_confirm_keeps_the_candidate_epoch_sub_slot_iters` — drives the real from-empty anchor confirm over loopback p2p (real mainnet block 5,000,000, real candidate record with the true epoch ssi `583,008,256`) and asserts the CONFIRMED record keeps the attested ssi. Red under the old fabrication (record came back `134,217,728`), green under the fix.

Consensus relevance: blocking for mainnet tip-follow — this was the recurring `Required iters … is not below the sp interval` wall. NOTE the open tail: crossing a LIVE epoch boundary (next: mainnet 9,146,880) with checkpoint-shallow history needs the weight-proof summary schedule substituted into the retarget itself (chia's `wp_summaries` path); tracked as follow-up work, not part of this fix.

DIVERGENCE-14 — time-lock conditions validated against the block's OWN height/timestamp instead of the previous transaction block's — FIXED

Status: FIXED on `full-node` commit `4296559`.

dg_xch path: `node/src/engine.rs` `add_block_with_delta` → `ConditionValidationContext { block_height, previous_transaction_block_timestamp }`.

chia oracle: `chia/consensus/block_body_validation.py` (lines 267–278) — the ASSERT/BEFORE height and seconds conditions are checked against the PREVIOUS TRANSACTION BLOCK's height and timestamp, never the containing block's own.

The live failure

The engine fed `block.height()` and the block's own foliage timestamp into condition validation. That over-rejects `BEFORE_*` conditions (the live stall: mainnet 9,143,056 rejected with `AssertHeightAbsoluteFailed`) and would under-reject `ASSERT_*` conditions at the boundary — both sides of the semantics were wrong, one of them visibly.

The fix

`add_block_with_delta` resolves the previous-transaction-block context before body validation: the parent record if it is a transaction block, else the record at `prev_transaction_block_height` from the store; `(0, None)` only at genesis. The pair threads through `derive_delta` → `validate_body` into `ConditionValidationContext`. Stored records were always correct — only the context read was wrong — so no resync was needed.

Proof: the previously-rejected mainnet block 9,143,056 validated and confirmed on the live node immediately after deploy (peak 9,143,055 → 9,143,057).

Consensus relevance: consensus-critical both directions: over-rejection halts sync on valid blocks; under-rejection would accept blocks chia rejects.

DIVERGENCE-15 — full header validation engages before the checkpoint window is slot-warm (`INVALID_ICC_VDF` at checkpoint+2) — FIXED

Status: FIXED on `full-node` (slot-warm gate, same commit as the sub-epoch-summary record machinery).

dg_xch path: `node/src/engine.rs` `derive_required_iters` (the full-vs-light validation gate).

chia oracle: `chia/full_node/weight_proof.py` `_validate_recent_blocks` — the recent-chain validator tracks slot/deficit state across the window and only fully validates blocks once that state is warm (past the first sub-slot boundary in the window); earlier blocks get the lighter proof-of-space treatment.

The live failure

After the DIV-13 fix, the fresh fast-sync (anchor 9,143,835) confirmed exactly two blocks and the FIRST strict-path validation — checkpoint+2, the earliest block whose prev and prev.prev are both cached — rejected a real mainnet block with `INVALID_ICC_VDF (invalid icc proof)` at 9,143,837, deterministically on every follow tick. The full validator's backward walks (challenge derivation, icc expectations, blocks-per-slot count) run to the previous SLOT START; at checkpoint+2 that walk leaves the known window, and the checkpoint-adjacent records carry cold-start deficit/challenge state (the headers-first walk warms them only at the first finished sub-slot). Note this wall was

UNREACHABLE before DIV-13: the poisoned ssi made earlier anchors fail at the sp-interval bound first. Each divergence fix exposes the next-deeper one — the ledger working as designed.

The fix

`derive_required_iters` gates the full path on chia's EXACT counted warm condition (`weight_proof.py _validate_recent_blocks`): more than **2 sub-slot boundaries** and more than **11 transaction blocks** accumulated in the cached ancestry below the parent (counted walking back; reaching genesis always allows the full path — the whole ancestry is known there). A first attempt gated on "one slot-start reachable" still rejected the block — one boundary is not enough context: the icc challenge walk terminates at an `is_challenge_block` record, i.e. on a DEFICIT comparison, and the checkpoint-cold deficits below the first boundary mislead it. With more than two boundaries below the parent, every strict walk terminates above the cold region. Until the gate opens, the ancestor-independent pospace path (the DIV-12 machinery) validates — the same treatment chia gives these blocks.

Offline reproduction (the proof)

`node/tests/icc_wall_9143837.rs` (env-gated on the uncommitted corpus at the builder pod's `/workspace/sync-corpus/icc-wall-9143837`: the real weight proof anchored at 9,143,835 plus the captured failing block range) drives the identical headers-first → in-order-confirm seam: under the old gate it reproduces `INVALID_ICC_VDF (invalid icc proof)` at 9,143,837 in under two seconds; under the counted gate the range confirms through 9,143,866.

Consensus relevance: blocking for mainnet tip-follow from any weight-proof checkpoint. Companion change (same commit): block records now carry their included sub-epoch summaries (headers-first attaches the proof-attested summary hash-verified against the header's declaration; the engine constructs-or-inherits with the same hash gate) — without ses presence `can_finish_sub_and_full_epoch` misfires once a boundary sits inside the record window and later new-slot blocks are spuriously rejected `INVALID_SUB_EPOCH_SUMMARY`, and the epoch retarget pass-throughs read the wrong values. Evidence: t050 `headers_first_attaches_the_weight_proof_summary_at_the_boundary` (real mainnet boundary slice + committed weight proof). Remaining OPEN tail: the first live EPOCH boundary (mainnet 9,146,880) needs the previous-epoch record walk — backfill to epoch depth is the follow-up.

DIVERGENCE-16 — consensus execution rejects unknown CLVM operators (`InvalidBlockSolution` at mainnet 9,143,859) — FIXED

Status: FIXED on `full-node` (Program::run strictness commit).

dg_xch path: `core/src/clvm/program.rs` `Program::run` — hardcoded `flags | NO_UNKNOWN_OPS` into EVERY execution, including consensus block-generator runs.

chia oracle: clvmr — `NO_UNKNOWN_OPS` is a MEMPOOL-mode flag. In consensus mode an unknown operator is a no-op with a well-defined cost (`op_unknown`, cost derived from the opcode encoding); only strict paths opt in. Same shape at the condition layer (chia_rs `parse_conditions`): `NO_UNKNOWN_CONDS` rejects unknown condition opcodes only in mempool mode.

The live failure

With DIV-15's warm gate in place, the offline checkpoint replay advanced past the icc wall and rejected mainnet block 9,143,859 — a transaction block whose generator (no back-refs) exercises the unassigned CLVM operator 28. chia executes it as an unknown-op no-op; dg_xch's forced strictness returned `ClvmError::Unimplemented(28)` → `InvalidBlockSolution`. The dialect already implemented `op_unknown` — the hardcoded flag made it unreachable from the generator path.

The fix

`Program::run` no longer forces `NO_UNKNOWN_OPS`; strictness is the caller's choice, mirroring clvmr. The strict callers were already explicit — `run_mempool_with_cost` passes `MEMPOOL_MODE` and the spend-bundle validator passes `NO_UNKNOWN_OPS | flags` — and the block-generator path passes the consensus flag set (no strictness), so consensus behavior now matches chia.

Evidence

The offline checkpoint replay (`node/tests/icc_wall_9143837.rs`, corpus-gated) confirms the full captured range 9,143,835..=9,143,866 — including 9,143,859, whose `validate_body` cost-equality check (execution cost must equal the block's declared `transactions_info.cost`) also pins `op_unknown`'s cost model against real mainnet data.

Consensus relevance: consensus-critical over-rejection: any block whose generator touches an unassigned operator halts the sync. Forward-compatibility too — post-softfork operators appear exactly this way to older validators.

DIVERGENCE-17 — `op_multiply` overcharges the running product held as i128 (`InvalidBlockCost` +456 at mainnet 9,143,994) — FIXED

Status: FIXED on `full-node` (limbs_for_num magnitude fix, this commit).

dg_xch path: `core/src/clvm/more_ops.rs` `limbs_for_num` — the `SExpNumber::I128` arm returned `i128::BITS.div_ceil(8)` = **constant 16**, the container width, regardless of the value held. `op_multiply` folds n-ary multiplies pairwise and re-derives `l0` (the running product's limb count) after every step; whenever the running product stayed in the i128 fast path, every subsequent step's linear term charged `16 * MUL_LINEAR_COST_PER_BYTE` instead of the value's true limb count.

chia oracle: clvmr 0.16.4 `more_ops.rs::limbs_for_int` — `v.bits().div_ceil(8)` over the value's **magnitude** bit length (num-bigint `BigInt::bits()` semantics). The running product's cost contribution follows the number, never its container.

The live failure

After DIV-16, live follow rejected mainnet block 9,143,994 with `InvalidBlockCost` — ours 5,973,350 vs declared 5,972,894, a +456 residual with every block-level cost component otherwise closing exactly.

The bisect

Isolation required three instruments, each diffed against a scratch clvmr 0.16.4 oracle binary running the identical generator spend:

1. **Per-spend cost diff** (chia_rs `run_block_generator2` oracle): one spend of coin `ae6f2f29…` carried the entire +456.
2. **Post-eval result trace** (pre_eval hooks on both engines, pair-program sites only): all 32,805 op-application results identical in value length and order — the divergence was invisible to results, pure cost accounting.
3. **Per-op-call cost trace** (delegating dialect wrapper on clvmr, matching trace on ours): exactly six `op 0x12` (multiply) calls differed — deltas +78, +78, +84, +216 — summing to precisely 456. Each delta decomposes as `(16 - true_l0) * 6` per fold step (e.g. true_l0=3 → +78, true_l0=2 → +84).

14 crafted micro-probes (divmod/mod/sub/mul/add/ash/lsh/concat/substr, negative and boundary operands) all matched — the defect only fires on n-ary multiplies with ≥3 operands, where an intermediate product's limb count enters the cost formula without ever being serialized.

Fix: `limbs_for_num`'s I128 arm computes the magnitude bit length (`128 - unsigned_abs().leading_zeros()`), `div_ceil(8)`, mirroring clvmr. Evidence: `core/tests/clvm.rs multiply_running_product_limb_cost_matches_clvmr` (two multiply vectors cost-pinned to clvmr 0.16.4: 2011 and 2962) and the offline corpus replay confirming through the 9,143,994 wall with exact cost equality.

Consensus relevance: blocking — any mainnet block whose generator performs an n-ary multiply with a small intermediate product fails cost validation.

Seam regression caught by the replay (prev-tx height 0 at the checkpoint anchor)

Keying the flag ladder off the previous-transaction-block height (chia's exact semantics) exposed a latent seam defect the old block-height keying masked: the checkpoint anchor confirms with `prev = None`, so its record stored `prev_transaction_block_height = 0`, and the first post-anchor transaction block derived `prev_tx = 0` → flag ladder at height 0 → genesis regime (`run_block_generator1`, no post-fork condition accounting) → `InvalidBlockCost` at 9,143,836, the first tx block after the anchor. The DIV-14 time-lock context shared the same collapsed value. Fix (DIV-13's candidate-threading pattern): at the anchor, both `confirm`'s prev-tx context and `compute_record`'s stored `prev_transaction_block_height` fall back to the headers-first candidate record,

which carries the weight-proof-attested prev-tx chain. Evidence: the corpus replay walks the full range 9,143,835 → 9,144,026 green, through both the seam block and the 9,143,994 cost wall, plus the cold-cache restart seam.

Companion changes in the same increment (chia 2.7.1 fork regime)

- `BlockGeneratorFlags::for_height` ported to the chia_rs 2.7.1 ladder keyed off **prev transaction block height**: `soft_fork8` (mainnet 8,655,000) → `DISABLE_OP | LIMITS` (op_modpow disabled until hard fork 2; mod/modpow operand caps); `hard_fork2` (unscheduled, 0xFFFFFFFA) → `COST_CONDITIONS | NEW_COST_MODEL` + keccak/secp outside guard.
- `op_mod` / `op_modpow` implemented with both old and new cost models, DISABLE_OP dispatch, and LIMITS operand caps (clvmr formulas verbatim; micro-probe `(mod (q . -7) (q . 3))` and modpow classes verified against the oracle).
- Soft fork 9 (`SIMPLE_GENERATOR | CANONICAL_INTS | LIMIT_SPENDS`, same mainnet height 8,655,000) is now **characterized and ported** — see "Soft fork 9 flag set" immediately below.

Soft fork 9 flag set (`SIMPLE_GENERATOR | CANONICAL_INTS | LIMIT_SPENDS`) — PORTED (this commit)

Status: PORTED on `full-node` (this commit). Resolves the DIV-17 companion note.

chia_rs `get_flags_for_height_and_constants` (docs.rs chia-consensus `spendbundle_validation.rs`, `if prev_tx_height >= constants.soft_fork9_height`) activates three flags at the soft-fork-9 height. Mainnet `SOFT_FORK9_HEIGHT = 8_655_000` (chia `chia/consensus/default_constants.py` `SOFT_FORK9_HEIGHT=uint32(8655000)`) — identical to soft fork 8; testnet override `3_924_000` (same file). dg_xch's `MAINNET.soft_fork9_height` matches both (`core/src/consensus/constants.rs`).

The three flags split by TYPE (chia-consensus `flags.rs`, `ConsensusFlags`):

1. **CANONICAL_INTS** (`0x0001`) — the ONLY member that is a clvm_rs VM flag (`ConsensusFlags::to_clvm_flags()` maps it 1:1 to `ClvmFlags::CANONICAL_INTS`; clvm_rs `src/chia_dialect.rs`). Effect: `uint_atom` (clvm_rs `src/op_utils.rs`) permits AT MOST one leading `0x00`, and only when the next byte's high bit is set (needed to keep the value positive); every other leading zero is rejected ("requires uN arg with no leading zeros"). When clear, all leading zeros are stripped (non-canonical accepted) — the pre-SF9 behavior. The ONLY VM consumer of this flag is the `softfork` operator's extension/expected-cost decode (clvm_rs `src/run_program.rs::parse_softfork_arguments`, `uint_atom::<4>` / `uint_atom::<8>`). It is NOT a cost change and NOT a condition-argument rule: condition-arg canonicalization is a SEPARATE, flag-INDEPENDENT check (chia_rs `crates/chia-consensus/src/sanitize_int.rs::sanitize_uint` takes no flag and always rejects non-canonical leading zeros — dg_xch already matches this, unchanged by SF9). CANONICAL_INTS is also permanently in clvm_rs `MEMPOOL_MODE` (`src/chia_dialect.rs`), i.e. chia's mempool always enforces it; dg_xch keeps it

strictly height-gated per the port directive, so pre-SF9 behavior (consensus AND the local mempool run) is byte-identical to today. The mempool-always-on stricter policy is a noted, deliberate deviation.

2. **SIMPLE_GENERATOR** (`0x0100_0000`, consensus-only — NO clvm effect) — enforces the simple generator shape: back-reference (generator-ref) lists banned, and the generator must be canonical serialization. Already enforced as a BODY rule in `validate_transaction_block` (`TooManyGeneratorRefs`, `is_canonical_serialization` -> `ComplexGeneratorReceived`), keyed on prev-tx height, matching chia `chia/consensus/block_body_validation.py` (`prev_transaction_block_height >= constants.SOFT_FORK9_HEIGHT` guards `TOO_MANY_GENERATOR_REFS` and `INVALID_TRANSACTIONS_GENERATOR_ENCODING`). No new code — cross-referenced here for completeness.

3. **LIMIT_SPENDS** (`0x0200_0000`, consensus-only — NO clvm effect) — caps a block at `MAX_SPENDS_PER_BLOCK = 6_000` spends. Already enforced as a BODY rule in `validate_transaction_block` (`TooManySpends`), keyed on prev-tx height. No new code.

Port: only CANONICAL_INTS required wiring, since the other two were already BODY-enforced. `BlockGeneratorFlags::for_height` (`core/src/consensus/block_generator.rs`) OR-s `CANONICAL_INTS` into `clvm_flags` at `height >= constants.soft_fork9_height`, extending the existing ladder in place (both block validation via `node/src/engine.rs` and the mempool run via `conditions_from_spend_bundle` inherit it). `op_softfork` (`core/src/clvm/more_ops.rs`) reads `dialect.flags()` and, when CANONICAL_INTS is set, rejects a cost atom whose leading `0x00` is not required for sign — the exact `uint_atom` canonical rule. dg_xch's `op_softfork` remains a simplified operator (it decodes only the cost via signed `int_atom`, ignores the extension, does not run the inner program or verify expected-cost against actual — a PRE-EXISTING divergence from clvm_rs `parse_softfork_arguments`, orthogonal to SF9 and tracked separately); this port faithfully implements the one SF9-gated slice (the canonical leading-zero rule on the decoded argument).

Keying (consensus-critical). chia keys the CLVM flag set on the PREVIOUS transaction block height — `get_flags_for_height_and_constants(prev_tx_height, ...)` in `chia/consensus/multiprocess_validation.py::_run_block` and `chia/consensus/blockchain.py` — while the generator VERSION keys on the block's OWN height (`block.height >= constants.HARD_FORK_HEIGHT`, same `_run_block`). This ladder keys the whole clvm set on the block's own height (DIVERGENCE-23, proven at the hard-fork boundary; the sibling soft-fork-8 `DISABLE_OP|LIMITS` share this key). The two keyings can differ ONLY at the single boundary block whose prev-tx straddles the activation height. For CANONICAL_INTS the sole on-chain consumer is the `softfork` operator, which real mainnet generators do not use, so the choice is observationally inert — the SF9 boundary corpus (era-e, just below 8,655,000, then across it) is the proof. If a future boundary block ever exercises a soft-fork clvm flag under a straddling prev-tx, the correct fix is a holistic re-key of the soft-fork clvm flags (SF8 and SF9 together) onto prev-tx height — deliberately deferred here to avoid re-opening DIV-23's locked, replay-proven own-height key.

Evidence: `core/tests/block_generator.rs` `height_flags_activate_canonical_ints_at_soft_fork9` (bit set at the height, unset below); `core/tests/clvm.rs` `softfork_canonical_ints_rejects_noncanonical_cost` (VM run: non-canonical

`0x00 0x05` softfork cost accepted with the flag clear, rejected with it set; canonical `0x05` accepted either way); the full consensus/engine/era-replay regression suite green; and the era-e SF9 boundary corpus validating across 8,655,000 with pre-fork blocks unchanged.

DIVERGENCE-18 — body validation rejects empty transaction blocks (`transaction block missing generator` at mainnet 9,144,185) — FIXED

Status: FIXED on `full-node` (this commit).

dg_xch path: `node/src/engine.rs` `validate_body` — hard-errored whenever a transaction block carried no `transactions_generator`.

chia oracle: `chia/consensus/block_body_validation.py` — the generator is OPTIONAL on a transaction block (an empty transaction block: reward claims, zero spends). With `transactions_generator is None`: the `generator_root` must be 32 zero bytes, the ref list must be empty (`generator_refs_root` = the 32×0x01 empty-list sentinel; a non-empty ref list without a generator is itself invalid), `conds` is None, declared `cost` must be 0, removals are empty, additions are the incorporated reward claims only, and the aggregated signature still verifies (over the empty spend set).

The live failure

Minutes after the DIV-17 deploy cleared 9,143,994, live follow walled at range 9,144,185..9,144,216 — block 9,144,185 is a real mainnet empty transaction block (`tx_block=true gen=false refs=[]`).

Fix: `validate_body` grows the generator-None arm mirroring chia's ladder exactly (zero-root, empty refs + sentinel root, cost 0, empty-set signature verify, reward-claims-only additions, `conds = None`).

Evidence: corpus extended with real mainnet ranges 9,144,027–9,144,216 (fetched via the chia full-node RPC oracle, size-verified). Red-first: the pre-fix engine reproduces the exact live wall at 9,144,185 offline; with the fix the checkpoint replay confirms the full range 9,143,835 → 9,144,216 green, now spanning the DIV-17 cost wall, the prev-tx seam, an epoch of icc slots, AND the empty transaction block.

Consensus relevance: blocking — empty transaction blocks occur routinely on mainnet whenever a transaction block interval passes with no spends.

DIVERGENCE-19 — genesis confirmed with a placeholder `required_iters = 0`, poisoning the era's challenge-block walk (`ip_iters` at mainnet height 36) — FIXED

Status: FIXED on `full-node` (this commit). First find of the genesis long-sync campaign (`--genesis-sync`, dg-xch-node-pg).

dg_xch path: `node/src/engine.rs` `derive_required_iters` — the genesis arm was `let Some(p) = prev else { return Ok(0) }`: the genesis block's proof of space was never validated and its record stored `required_iters = 0`.

chia oracle: `validate_unfinished_header_block`'s genesis branch validates the genesis proof of space like any block — challenge from the genesis constant, difficulty the genesis weight itself — and stores the real `required_iters`. `calculate_ip_iters` rejects `required_iters == 0` unconditionally (pot_iterations), so a zero can never appear in a valid record.

The live failure

The very first genesis-sync run validated mainnet blocks 0–35 and walled at height 36 with `io error: ip_iters`. Block 36 is the first block that opens a new sub-slot (1 finished sub-slot); its icc scaffold walks back to the era's challenge block — the genesis record — and `curr.ip_iters(constants)` explodes on the stored zero.

The bisect

Offline repro in one pass: real mainnet blocks 0–1023 (chia full-node RPC oracle, size-verified) through the from-empty follow harness (`node/tests/genesis_wall_36.rs`) — wall reproduced at 36 in 23s. One instrumented run named the record exactly: `site=curr224 h=0 ssi=134217728 spi=2 ri=0` — the genesis record, required_iters zero.

Fix: the genesis arm computes real `required_iters` by validating the genesis proof of space (`pospace_required_iters_at` with difficulty = the genesis weight, since there is no prev to subtract). Evidence: the corpus replay confirms blocks 0..=1023 green (the red-first wall at 36 flipping).

Consensus relevance: blocking for any from-genesis sync; invisible to checkpoint sync (the anchor is never height 0 and the walk never reaches a genesis record).

Characterized follow-up (OPEN): genesis header validation is still the pospace-light path — the genesis VDF chain (challenge → cc/rc VDFs from the genesis challenge) is not yet fully validated the way chia's genesis branch does. Tracked for the genesis-campaign hardening pass.

DIVERGENCE-20 — aggregate verification omits bundle-level `AGG_SIG_UNSAFE` pairs (`BadAggregateSignature` at mainnet 2,272,201) — FIXED

The wall

First fruit of the era-targeted replay campaign: the era-a corpus (mainnet 2,269,999–2,278,191, the chain's heaviest 2022 window, extracted directly from a chia node's database) anchored and confirmed 2,202 blocks of full body validation, then rejected block 2,272,201 — a real mainnet block — with `BadAggregateSignature`. Notably this was the node's first-ever contact with a real `AGG_SIG_UNSAFE` condition: the genesis sync had not yet reached the transaction era, and every block fixture in the tree carried only `AGG_SIG_ME`.

The bisect

Standalone probe (`node/tests/div20_probe.rs`): the generator run is byte-exact (64 spends, cost 3,350,725,246 matching `ti.cost`), the pair population is 54 `agg_sig_me` + **2 `agg_sig_unsafe`**, and the aggregate rejects. In chia_rs, `AGG_SIG_UNSAFE` pairs live on `SpendBundleConditions` itself (they sign their raw message, bound to no coin), not on any spend. `validate_block_aggregate_signature` assembled its pair set exclusively from per-spend fields — the two bundle-level pairs were silently absent, so the pairing product could not match a signature that includes them.

Fix: the verifier folds `conds.agg_sig_unsafe` into the key/message set ahead of the per-spend pairs (mirroring chia_rs `pkm_pairs`). Red-first unit: an `AGG_SIG_UNSAFE`-only bundle signed with a real key must verify (`agg_sig_tests::bundle_level_agg_sig_unsafe_pairs_join_the_aggregate`); evidence: the era-a replay walks through 2,272,201 after the fix.

Consensus relevance: blocking for every block containing an `AGG_SIG_UNSAFE` condition — common from the 2022 era onward. Invisible to tip-follow only until the first such block arrives.

DIVERGENCE-21 — consensus enforced the mempool-only 1024-announcement cap (`InvalidCondition` at mainnet 4,693,324) — FIXED

The wall

The era-b corpus (mainnet 4,689,999–4,698,191, the modern transaction band) confirmed 3,325 blocks and rejected 4,693,324 — an 830-spend dust-sweep block chia accepted — with `InvalidCondition`.

The bisect

The standalone probe reruns the block byte-exact (cost matches `ti.cost`, aggregate signature verifies) and isolates the rejection to `validate_block_conditions`: the running announcement count crossing `ANNOUNCEMENT_LIMIT = 1024`. That cap is chia's MEMPOOL rule (chia_rs `TOO_MANY_ANNOUNCEMENTS`, mempool mode only) — added during the dust storm precisely because on-chain blocks exceed it. There is no announcement-count consensus rule anywhere in chia's block validation, and mainnet block 4,693,324 existing at all is the chain's own proof.

Fix: the consensus path drops the cap (the mempool-side spend-bundle check keeps it). Red-first: a conditions set with >1024 announcements must pass block validation; evidence: the probe flips green and the era-b replay continues past 4,693,324.

Consensus relevance: blocking for announcement-heavy blocks — routine from the dust-storm era onward in the tx-dense bands.

DIVERGENCE-22 — sign-pad corruption in BigInt atom encoding (era-c wall 5,494,140)

Real mainnet block 5,494,140 (an offer-settlement block: 12 payment announcements against 11 settlement spends plus one more) rejected with `AssertAnnounceConsumedFailed`.

The bisect

Announcement census: 25 creates, 24 asserts, exactly 2 asserts with no creator and zero near-misses. chia_rs oracle (`run_block_generator2`, identical spend identities byte-for-byte) proved the announcement atoms themselves diverged. Extracting spend 4's puzzle + solution and running both VMs on identical bytes: identical cost (210,674), two conditions differ. sha256 call tracing on both VMs found the first divergent leaf: our VM hashed the atom `c0cecd48c0` where clvmr hashed `00cecd48c0`. The value (3,469,560,000 = `0xCECD48C0`, a multiply result on the BigInt path) was computed CORRECTLY — the corruption is in `formatting::bigint_to_bytes`: for a positive value whose magnitude fills its u32 words exactly (minimal encoding 4k data bytes with the high bit set), the sizing adds a sign-pad byte but the tail-write loop re-reads the already-consumed top u32 word and overwrites the pad with the value's LOW byte. `00 CE CD 48 C0` became `C0 CE CD 48 C0`. The boundary sweep also caught `−2^31` (negative 4-byte boundary) encoding wrong. The bug only bites the BigInt path — atoms ≤16 bytes ride the correct `i128` encoder — which is why mainnet survived to 5,494,140: this puzzle multiplies against a 32-byte zero-padded operand, forcing BigInt.

Fix: `bigint_to_bytes` rewritten to the canonical encoding — `to_signed_bytes_be` + minimal sign-byte strip (chia's `int_to_bytes`). Red-first: `core/tests/bigint_encode.rs` (`div22_pad_byte_regression` + a ±2^k boundary sweep against the reference, round-tripped through `number_from_slice`); evidence: the spend-4 VM diff flips to byte-identical with clvmr (conds 22/23 = `49a1…`/`078e…`, cost equal) and the era-c corpus replay crosses 5,494,140.

Consensus relevance: any block whose CLVM emits a computed integer atom in the corrupt ranges through the BigInt path — offers/CAT price math is the natural source. Blocking for era-c and beyond.

DIVERGENCE-23 — CLVM flag ladder keyed on prev-tx height, not the block's own (era-c wall 5,496,002)

First transaction block past the hard fork (5,496,000) rejected with `InvalidBlockCost`: our cost 411,338,833 vs ti.cost 398,155,295.

The bisect

The probe showed identical spends and a cost overcharge exactly at the fork boundary. chia_rs oracle: `run_block_generator2` on the same generator = 398,155,295 = ti.cost — chia validated the block with POST-fork accounting even though its previous transaction block (5,495,999) is pre-fork. The ladder keys on the block's OWN height. Our prev-tx keying was indistinguishable everywhere except at a flag boundary whose first tx block has a pre-boundary prev tx — exactly this block.

Fix: `BlockGeneratorFlags::for_height` takes the block's own height; the precompute guard (prev-tx key match) is deleted — a precomputed body is now always valid since its flag key is fully known at precompute time. Evidence: era-c replays through the hard fork end-to-end (12.53 blocks/s); genesis, era-d, and era-a (14.07 blocks/s) unchanged — below any boundary both keyings produce identical flags, so only boundary blocks could regress, and era-c holds the only one.

Consensus relevance: blocking at every flag-activation boundary (hard fork 5,496,000, soft-fork 8, hard-fork 2) — one wrong-regime block each, then permanent divergence.

DIVERGENCE-24 — CLVM `point_add/pubkey_for_exp`: invalid-point and cost-timing semantics diverge from clvmr — OPEN (characterized)

Surfaced while porting the two BLS operators from bls12_381 to blst (the dependency-normalization change that removed the vestigial `bls12_381` dependency). The port is behavior-preserving — a byte-level differential against the previous bls12_381 implementation (`core/tests/bls_ops_differential.rs`, ~10,000 seeded cases) proves identical cost and output bytes — but characterizing the operators against the reference exposed three deliberate dg_xch deviations from clvmr 0.17.7 (the clvm_rs chia mainnet runs via chia_rs 0.42.x, `src/more_ops.rs`):

1. **`point_add` silently skips an invalid 48-byte G1 encoding** (no cost, no error) — the in-code comment says "parity with the previous implementation". clvmr's `Allocator::g1` (chia-bls `G1Element::from_bytes`) errors the whole operator on any invalid encoding (non-canonical, x >= p, off-curve, wrong subgroup, bad infinity).

2. **`point_add` charges `POINT_ADD_COST_PER_ARG` only for valid points, after parsing.** clvmr charges every argument *before* parsing and checks `max_cost` per argument, so a tight budget with an invalid argument is `CostExceeded` in clvmr and `Ok` here.
3. **`pubkey_for_exp` never checks `max_cost`** (`_max_cost` unused). clvmr `check_costs` the exponent-priced cost before the scalar-mul.

Reference: clvmr 0.17.7 `op_point_add` / `op_pubkey_for_exp` (`src/more_ops.rs`), chia-bls 0.42.1 `PublicKey::{from_bytes, from_integer}` (`crates/chia-bls/src/public_key.rs`). chia_rs pin: chia-blockchain `pyproject.toml` `chia_rs >=0.42.1` -> `clvmr = "0.17.7"`.

Tests: `core/tests/bls_ops_differential.rs` — `point_add_rejects_invalid_point_like_clvmr`, `point_add_charges_per_arg_before_parse_like_clvmr`, `pubkey_for_exp_enforces_max_cost_like_clvmr`, each `#[ignore = "DIVERGENCE-24: ..."]` and encoding *chia's* semantics; removing the `#[ignore]` reproduces the divergence.

Consensus relevance: medium-high, latent. No historical mainnet block can contain an invalid point in a `point_add` (chia would have rejected the block), so era replays cannot surface it — but a *future* adversarial block built to be rejected by chia and accepted here (garbage point, or a cost-boundary generator) is a chain split. The fix must land as its own change with a fresh differential + era gate, not inside the dependency port (which is required to be bit-for-bit preserving).

DIVERGENCE-25 — pospace BitReader under-counts the 64-bit field span for k>32 metadata, rejecting valid proofs (`Values Not in Same Group` / `INVALID_POSPACE`, live mainnet 6,281,496) — FIXED

Live pi-IV (`full-node --sync-from=6000000`, mmap store) walled at 6,281,496, retrying forever with `Failed to Validate Proof: Custom { kind: InvalidData, error: "Values Not in Same Group" }` (`proof_of_space/src/lib.rs:109`) surfaced as `INVALID_POSPACE`. Mainnet is far past this height, so the block is valid and we mis-rejected it — a deterministic wall, not a bad peer.

The bisect

The fixture is the real block, exported from a chia `blockchain_v2_mainnet.sqlite` (`blk_6281496`, `in_main_chain=1`, zstd `block` column) and replayed through the exact node call path. **pos.size = 41** — the winning proof is a k41 plot (proof = 64 * 41 bits = 328 bytes), the first k>32 proof to reach this leg. The earlier gates all pass (`calculate_pos_challenge` matches `pos.challenge`, plot filter passes), so `plot_id`/challenge/signage-point are genuine; the failure is purely in our decode of the proof bytes. `uncompress_proof` and the f1 layer (`get_proof_f1_and_meta`) succeed; the fault is in `forward_prop_f1_to_f7` metadata extraction. A debug build panics at `utils/bit_reader.rs:128` (`field >>= 64 - last_field_bits`, shift by >= 64); the

pi's release build instead wraps that shift, producing corrupt table metadata, which then fails the (correct) `fx_match` adjacent-bucket test and raises "Values Not in Same Group". Same root cause, two surfaces.

The matching primitive is NOT the bug: `fx_match` + `L_TARGETS` (`plots/fx_generator.rs`, `constants.rs`) are a byte-exact port of chiapos `calculate_bucket.hpp` `load_tables/FindMatches` (`(((indJ+m)%kB)*kC + (((2m+parity)^2+i)%kC)`, adjacent-bucket + parity indexing). The fault is in dg_xch's own bit-slicing helper `BitReader::from_bytes_be_offset`.

Root cause

`from_bytes_be_offset` computed the field span as `end_field = (start_field*64 + bit_size) >> 6`, dropping the intra-field `bit_offset % 64`. When `(bit_offset % 64) + (bit_size % 64)` crosses a 64-bit word boundary, the value spans one more u64 field than `field_count` accounts for, and `last_field_bits = (bit_size + bit_offset) − (field_count−1)*64` overshoots 64. `fx_gen` calls this for tables 4-6 with `bit_offset = (k+6) mod 8` and `bit_size = k * meta_multiplier`. For **k32** the offset is 6 and widths are 32-multiples, so the span never crosses a boundary — `end_field` is identical either way for every multiplier, which is why 281,496 blocks validated. For **k41** (offset 7, e.g. the 3k = 123-bit metadata field) the buggy form gives `field_count = 2, last_field_bits = 66`; the correct form gives `field_count = 3, last_field_bits = 2`.

Fix: `end_field = (bit_offset + bit_size − 1) >> 6` — the field index of the LAST bit, using the ABSOLUTE offset. This is generic bit arithmetic (no chia-specific constant); it is provably a **no-op for k32** (`buggy == fixed` across all metadata multipliers) so no k32-era replay can regress, and it keeps `last_field_bits` in `[1,64]` for all k. Red-first: `proof_of_space/tests/div25_pospace_6281496.rs` replays the block's k41 pos primitives (`k || plot_id || challenge || proof` fixture); pre-fix it panics/mis-decodes, post-fix `validate_proof` returns the real non-zero quality `0x515fac0a1c958a26856dfeae617220c7f8ce6e5552fe68fd59b1fb6c3c2e2002`. Evidence: the pospace suite is green and clippy is clean.

Origin note: this is a defect in our reimplementation, not a chia bug — chiapos (C++) slices these bits correctly — so there is no upstream fix to mine; the divergence is ours and appears only once a k>32 plot wins a block.

Consensus relevance: blocking for ANY block whose winning proof comes from a k33-k50 plot. Rare (k32 dominates netspace) but permanent: a pure validating node walls at the first such block — mainnet 6,281,496 is the first to reach this leg.

DIVERGENCE-26 — the live add-block path skipped chia's coin-store body validation (found by audit) — FIXED

Found by the Tier-0 parity audit, not a live wall:
`node/src/engine.rs::validate_body` validated the generator identity (gen_root/refs_root), cost, the aggregate signature, and the block-level conditions — but with an **empty `coin_context`** and **no coin store lookups**. Core's faithful

`validate_transaction_block` (`core/src/consensus/block_generator.rs`) had zero callers in `node//full-node/`. Missing on the live path, per `chia/consensus/block_body_validation.py` (CNI 2.7.1):

- **Rule 15** — removals exist & unspent (`UNKNOWN_UNSPENT` / `DOUBLE_SPEND` / `DOUBLE_SPEND_IN_FORK` via ForkInfo)
- **Rule 16** — no minting (`removed < added` over ACTUAL stored amounts)
- **Rules 17/18/19** — reserve-fee coverage, fee+farmer-reward cap, declared fee == computed fee
- **Rule 20** — removed coin's stored puzzle hash matches the spend's reveal (reachable only via a corrupt store row — the coin id commits to the puzzle hash — but chia checks it and now so do we)
- **Rule 11** — foliage additions/removals merkle roots vs the actual delta
- **Rule 12** — foliage BIP158 transactions-filter hash (the producer already built the identical filter; validation reuses `chia_block_filter`)
- **Rule 3** — `foliage_transaction_block.transactions_info_hash` binding
- **Rule 10** — coin amount bounds; unified rules 13/14 duplicate checks
- **Rule 5** — reward-claim correctness (the engine previously TRUSTED `ti.reward_claims_incorporated` wholesale)
- **Rule 21 coin context** — with `coin_context` always empty, `validate_spend_context` (birth-height/seconds + relative time-locks against the SPENT coin) never ran.

Where chia runs this, and where we now do

chia runs `validate_block_body` for **every** block added: the singleton path (`full_node.py::add_block` → `Blockchain.add_block`) and the long-sync batch path (`add_block_batch` → `add_prevalidated_blocks` → `Blockchain.add_block`) both land on it; `skip_blocks` only skips blocks already validated below the fork point. There is **no body-validation skip window** in chia 2.7.1. The engine mirrors that: the rules run on every transaction block on both `add_block` and the staged-window path (`Engine::validate_body_coin_rules`, called from `prepare_delta` before record derivation), at one batched `get_coin_records` per transaction block.

Two deliberate dg_xch-only gates (chia has neither mode):

- **Anchored stores** (`--sync-from`): no coin set and no records exist below the anchor, so the store-backed rules (15-21 context) are undefined there. They key on a lazily-resolved full-history flag (`min_record_height == 0`, or an empty store whose first block is genesis; `Engine::with_enforced_coin_rules` forces strict). The pure structural rules (3, 10, 11, 12, 13, 14) run on EVERY transaction block regardless; rule 5 runs whenever its record walk resolves (always under full history, and in practice for the whole era-corpus window above an anchor).
- **Assume-valid (T055)**: continues to skip only the BLS verify and the script-output condition checks below the milestone. The structural body bindings (rule 3/4 chain) stay ON below the milestone — they are what bind the body content to the WP-attested header, so bypassed state stays exact (the t055 fixture now corrupts the signature and REBINDS the foliage hashes, keeping the violation BLS-only).

ForkInfo equivalent

chia's `ForkInfo` tracks additions/removals along a fork branch so re-applied blocks validate against the fork's coin set. The engine equivalent (`ForkView`) walks parent-ward through the unapplied deltas — `staged_deltas` (window blocks whose coins commit at window confirm; new overlay, drained at confirm) then `pending` (orphan branches, reorg horizon-bounded) — down to the last store-applied ancestor (`fork_height`). Main-chain rows confirmed/spent above the fork point are ignored for the branch, exactly as chia's rule 15 interprets `fork_info`. A branch whose deltas have been pruned beyond the reorg horizon is rejected with an explicit error (chia would rebuild via `advance_fork_info`; the engine cannot reorg there either).

Red-first evidence

`node/tests/t070_body_coin_rules.rs`. On the pre-fix engine every red test's invalid block was fully ACCEPTED (`Extended {5000004}`): double spend, unknown coin, tampered reward claims / fees / roots / filter / ti-hash, minting, fork double-spend, wrong birth-height assert. Post-fix all reject with chia's exact error, and two false-positive guards pass: real mainnet block 5,000,004 under FULL enforcement (removals seeded unspent at their real heights, reward-claim walk grounded by sub-fixture records carrying the block's own claim puzzle hashes — proving `create_pool_coin`/`create_farmer_coin` and the BIP158 filter byte-match mainnet), and an honest synthetic post-hard-fork spend. Corpus gates rerun green with enforcement live: genesis-0-1023 (full enforcement from height 0), era-c 5,493,999 (6,144 real blocks across the hard-fork straddle), era-e 8,653,999 (soft-fork-9 era).

Consensus relevance: blocking-severity gap class — without rule 15 a malicious peer's block spending nonexistent/spent coins would corrupt the coin set; without rules 5/16/19 it could mint value or forge reward claims. Found and closed by audit before any live wall.

DIVERGENCE-27 — unfinished blocks relayed after header-only validation: the generator never ran (live 600s GENERATOR_RUNTIME_ERROR ban, 2026-08-20) — FIXED

Found by the Tier-0 parity audit and root-caused against a live ban: at 2026-08-20 19:50 mroneleg's chia 2.7.1 node banned us for 600 seconds with `GENERATOR_RUNTIME_ERROR` (Err 117) after fetching an unfinished block we had relayed.

chia (CNI 2.7.1): `full_node.py::add_unfinished_block` runs, in order: the disconnected-parent check → the pos-quality check → `seen_unfinished_block` (the exact-UB-hash dedup set — "This is intentional, to prevent DOS attacks") → `get_unfinished_block2` (already held / better variant held) → the peak `total_iters` and `MAX_SUB_SLOT_BLOCKS` drops → header validation (`validate_unfinished_block_header`, which also enforces the structural generator/`transactions_info` bindings: generator root, refs, `transactions_info` hash, data-presence — blockchain.py:713-734) → **the generator EXECUTION** (`_run_block` = `run_block_generator(2)` with refs from `lookup_block_generators`

against the parent's branch, budget `min(MAX_BLOCK_COST_CLVM, transactions_info.cost)`, aggregate signature validated inside the run; full_node.py:2497-2536) → `validate_unfinished_block` (full `validate_block_body` with the conditions, including the cost rules 7/9) — ALL before `full_node_store.add_unfinished_block` and the `NewUnfinishedBlock/2` broadcast (full_node.py:2570, 2636-2656). Any of these raising `ConsensusError` bans the SENDER for `CONSENSUS_ERROR_BAN_SECONDS` = 600 (protocol_timing.py, ws_connection.py:610-614); Err 117 covers a generator that fails to deserialize or any non-cost `EvalErr`.

dg_xch (pre-fix): `full-node/src/daemon.rs::process_ub_inbox` validated a received unfinished block through `validate_unfinished_header_block` ONLY — the generator fields were never read (the `UnfinishedHeaderBlock` projection drops them) — then cached it in the served `unfinished` cache and queued the `NewUnfinishedBlock2` announce. Honest peers fetched the poisoned block from our cache (p2p `RequestUnfinishedBlock/2`), ran the generator themselves, hit the error chia never lets past `add_unfinished_block`, and banned US. No dedup ran on the inbox either (`UnfinishedCache::seen` existed with zero callers), so duplicate announces re-validated per copy.

Fix (`full-node/src/daemon.rs`, `node/src/engine.rs::validate_unfinished_block_body`): after header validation and BEFORE the cache/announce, every unfinished block — peer-received and own-farmed alike, exactly as chia's `farmed_block=True` path — now passes the transactions gate: structural bindings (`transactions_info` hash, generator root, refs root, data-presence), SF9 rules (ref-list ban, canonical serialization, spend cap; keyed on the previous transaction block's height), the generator RUN with chia's clamped budget `min(MAX_BLOCK_COST_CLVM, claimed)` (so an understated claim fails DURING the run after burning at most the claimed cost — chia's DoS bound), exact cost equality, and the aggregate-signature verify. Failures drop the block (metrics reasons `ub_generator_fail` / `ub_cost_mismatch` / `ub_body_fail` / `ub_generator_ref_missing`), never cache, never relay. chia's dedup ladder now fronts the pipeline: the `seen` exact-hash set plus the `get_block2` already-held/better-variant check, both after the disconnected-parent park (chia's order — a "we are behind" block is not seen-marked and re-processes after catch-up), so a burst of duplicates costs ONE header validation and ONE generator run.

Deliberate deltas (documented, not gaps):

- **Ban posture:** chia bans the sender 600s on `ConsensusError`. Our p2p layer has no timed ban list and the `RespondUnfinishedBlock` inbox does not carry the sender's peer id; the enforceable action is the drop + no-relay, which closes the harm vector. Sender punishment lands with the p2p ban list (the posture already documented at `p2p/src/handlers.rs` for `TransactionAnnounceAction::Ban`).
- **Ref resolution:** chia's `lookup_block_generators` resolves against the parent's branch; ours resolves the main chain via `get_generator_at_height`. At the live tip these coincide, and past SF9 (mainnet 8,655,000) ref lists are banned outright, so the walk is near-dead code on mainnet.
- **Cost-exceeded error code:** an understated claim surfaces as `BlockCostExceedsMax` from the clamped run (chia: same), an overstated one as `InvalidBlockCost` from the equality rule (chia: same) — codes match; our deserialize failure maps to `InvalidBlockSolution` where chia_rs reports 117, observationally identical (both drop, neither relays).

Residual (out of this fix's scope, tracked): chia's `validate_unfinished_block` also runs the full coin-store body rules (`validate_block_body` rules 15-21 — double spend, unknown coin, fees) on unfinished blocks; our unfinished path runs the structural + execution + cost + signature set but not the coin-store rules (they run at finished-block confirm, DIVERGENCE-26). A UB whose generator runs clean but spends an unknown coin would still be relayed — a remaining (narrower) ban-vector species. Likewise chia's pos-quality, peak-`total_iters`, and `MAX_SUB_SLOT_BLOCKS` pre-checks are not yet mirrored on the inbox path.

Red-first evidence: `full-node/src/daemon.rs::ub_relay_gate_tests` — the real mainnet recent-chain slice (9,054,336..=9,054,620) rebuilt into light-path records so projected UnfinishedBlocks pass REAL header validation (real pos, real plot-key foliage signatures, real VDFs) and reach the relay decision. Pre-fix, a header-valid block with a bogus generator spliced in was CACHED and ANNOUNCED (all three no-relay assertions failed; duplicates produced two announces); the honest no-generator control passed pre- and post-fix (no false positive). Post-fix all four pass. The root-consistent execution species — unforgeable through the daemon harness because the plot-key foliage signature commits to `transactions_info` — are proven at the pure-fn level in `node/tests/unfinished_body.rs` against real mainnet block 5,000,004: undeserializable generator → `InvalidBlockSolution`, CLVM raise → `GeneratorRuntimeError`, cost overstated → `InvalidBlockCost`, cost understated → `BlockCostExceedsMax` during the clamped run, honest block → exact-cost pass with the real aggregate signature, non-tx fast path untouched.

Consensus relevance: blocking-severity network-behavior gap — every relay of a generator-invalid unfinished block converts each honest fetching peer into a 600s ban of us, degrading our peer set exactly when a malicious farmer floods bad partials. Closed at the source: nothing unvalidated is served or announced.

DIV-HINT — `coin_hint` index stores only 32-byte hints (deliberate)

Status: deliberate design divergence in table CONTENTS; query behaviour is byte-identical to chia. Not a bug, never `#[ignore]`d.

chia:
`chia/full_node/hint_management.py::get_hints_and_subscription_coin_ids` stores `(coin_name, hint)` for every non-reward create-coin whose hint is `0 < len(hint) <= 32` — i.e. hints of 1..=32 bytes land in the `hints` table.

dg_xch: `coin_hint` is a fixed 32-byte-keyed table (`Bytes32` hint column), so `hints_for_conditions` (core/src/consensus/block_generator.rs) indexes a create-coin hint only when it is EXACTLY 32 bytes; shorter hints are skipped.

Why this is behaviourally identical: chia's entire hint READ/subscription surface is 32-byte. `get_coin_records_by_hint` takes a `bytes32` (full_node_rpc_api.py:812), and a puzzle-hash subscription fires only at `len(hint) == 32` (`hint_management.py`: `if len(hint) == 32`). A hint that is not exactly 32 bytes is therefore unreachable through any chia query path — it is dead weight in chia's table. dg_xch simply never writes that dead weight, so every query returns the identical coin set. No variable-length `BLOB` migration is warranted (approved: parity item #3).

Consensus relevance: none — hints are a wallet/explorer service index, not a consensus input. The `coin_hint` table populates on block apply in the SAME transaction as the coin deltas (`CoinStore::apply_hints_in`); a store synced before this landed needs a one-time hint reindex (walk confirmed blocks, re-run the generator, re-derive hints) — there is no in-place backfill.

DIVERGENCE-28 — peak selection ignored weight: one global height-max slot, unverified, never retracted (found by Tier-0 audit) — FIXED

Found by the Tier-0 parity audit, not a live wall: `full-node/src/daemon.rs::on_new_peak` recorded a peer's `NewPeak` as `claimed_peak.fetch_max(peak.height)` and stored the tip hash only on a strict HEIGHT advance — one global slot, no weight comparison, no per-peer tracking, no verification of the claim, and no rollback. Four consequences:

1. a longer-but-LIGHTER fork became the sync target (height beat weight — the opposite of Chia fork choice);
2. two peers racing competing peaks resolved by height, not weight;
3. a peak whose weight proof failed was re-selected every driver tick;
4. a false/withdrawn announcement (peer over-claims, then disconnects or never serves it) pinned `claimed > local` forever, grinding the FOLLOW band.

chia (CNI 2.7.1): `full_node.py::new_peak` records the announcing peer's `(header_hash, height, weight)` claim FIRST (`sync_store.peer_has_block` → `peer_to_peak`, one claim per peer, newest announcement replaces), then drops announcements not heavier than the local peak ("Not interested in less heavy peaks", `peak.weight > request.weight → return`). The long-sync target is the HEAVIEST collected claim (`sync_store.get_heaviest_peak`, :105). The claim is VERIFIED before it is synced to: `request_validate_wp` (:1154-1159) cross-checks the weight proof's recent-chain tail against the claimed height AND weight and closes the peer on mismatch or failed validation; a peak that failed is held in `bad_peak_cache` (:175, :3357-3390, cap 100, min-height eviction) and `in_bad_peak_cache` refuses any proof riding through it. A disconnect retracts the peer's claim (`on_disconnect` → `sync_store.peer_disconnected`, sync_store.py:121).

dg_xch now: `full-node/src/peak_book.rs` (`PeakBook`) holds per-connection claims (replace-on-announce, cap 256 = chia's `peak_to_peer` bound), heaviest-claim selection, disconnect retraction, and the bad-peak quarantine (cap 100, min-height eviction). `Node::sync_target` = heaviest selectable claim, gated strictly heavier than the local peak's weight; every band (event-driven tip_follower, the FOLLOW fill target, the driver's caught-up check and fast-sync trigger) consumes it, so a lighter-but-longer fork idles every band instead of grinding one. `validated_proof` cross-checks the fetched proof's recent-chain tail against the claimed (height, weight), quarantines the peak and stops the serving peer on mismatch or failed validation, and refuses proofs riding through a quarantined hash. Retraction is structural where chia's is a callback: outbound claims key on a per-connection `ClaimGuard` whose `Drop` (with the connection's handler map) retracts; inbound claims key on the real peer id and reconcile against the live inbound map each driver tick; a 300 s liveness TTL backstops both (honest peers re-announce per network peak, ~18.75 s). When NO peer serves a weight proof for a claimed tip, every claim on that hash is retracted (the soft analog of chia closing the non-serving peer) so a

phantom peak cannot be re-fetched tick after tick. The `claimed_peak` gauge is now the published heaviest-claim height and ROLLS BACK on retraction/quarantine (`tip_lag` stays truthful).

Deliberate deltas from chia, both stricter: (a) chia 2.7.1's `add_to_bad_peak_cache` is dead code in production (only the CHIP-0013 batch-sync path ever called it; that call was dropped in the message-conditions refactor #17772) — we quarantine on ANY weight-proof mismatch/validation failure for the claimed tip; (b) chia tolerates equal-weight claims at the announcement gate but refuses them at `request_validate_wp` ("already caught up") — our single `sync_target` gate is strictly-heavier throughout, which cannot lose a reorg (equal weight never reorgs chia either).

Proven red-first: `peak_selection_prefers_weight_over_height` and `withdrawn_claim_retracts_when_the_announcing_connection_drops` (daemon tests) fail on the fetch_max slot and pass on the book; `sync_target_weight_gates_against_the_local_peak` pins the weight gate + no-reselection against the real mainnet 5,000,000 record; `peak_book` unit tests pin replace-on-announce, retraction, reconcile, quarantine, and both cache bounds.

Consensus relevance: sync-target choice only — block validation and fork choice inside the engine were already weight-correct; this fixes WHICH peak the daemon decides to sync toward and whether a bogus claim can wedge that decision.

DIVERGENCE-29 — the reorg was not atomic: N separate store transactions where chia holds one (found by Tier-0 audit) — FIXED

chia 2.7.1: the ENTIRE add-block-with-reorg state transition executes inside ONE database transaction — `async with self.block_store.transaction():` (chia/consensus/blockchain.py `add_block`) wraps `add_full_block` and `_reconsider_peak`, i.e. `coin_store.rollback_to_block` (coin_store.py), per-branch-block `coin_store.new_block`, the hint writes, and the block store's main-chain flips + `set_peak`. A crash anywhere in that span means the reorg never happened; `transaction()` is `db_wrapper.writer()` (block_store.py).

dg_xch before: `Engine::reorg` (node/src/engine.rs) ran `rollback_to` in its own transaction, then per-branch-block `apply_block` + `apply_hints` each in their own, then `set_peak` in its own. A crash between the first and last transaction left the store TORN: coins reverted above the fork while the peak still pointed at the old branch — and nothing on boot reconciled. All three backends were exposed; the Postgres pool's `synchronous_commit = off` widened the window (a durable rollback with the re-apply suffix lost as a clean tail). The linear path was already atomic (`apply_on_chain` folds coins + hints + peak into the one open per-block batch); only the reorg streamed.

dg_xch now: `Engine::reorg` threads ONE open batch through the whole transition — a new `CoinStore::rollback_to_in` (batch variant, the `apply_hints_in` no-`Self:Sync`-leak precedent), then per-branch-block `apply_block_in` + `apply_hints_in`, then `set_peak_in`, one `commit`. Per backend: - **SQLite:** all statements share the batch's BEGIN..COMMIT on the single writer connection; a dropped batch is rolled back by `begin()`'s guard. - **Postgres:** the batch is one transaction; `rollback_to_in` additionally issues `SET LOCAL synchronous_commit = on` — a reorg is rare (fsync cost irrelevant) and must be durable, unlike the resyncable per-block stream the pool-wide async-commit

posture covers. Atomicity holds either way (async commit loses only clean suffixes); the SET LOCAL buys durability. - **mmap (no transactions):** `apply_block_in` now defers EVERY table mutation (spends included — previously write-through) as staged absolute payload images, and `rollback_to_in` stages the fork sweep; a dropped batch leaves only unreferenced log frames. The publish itself (inside `set_peak_in`) is bracketed by `reorg.journal`, written durably before the first in-place mutation and removed after the peak-last meta write; a journal found at open CONVERGES the store back to the fork (coins, main flags, heights, peak — all idempotent, re-crash safe). A journal surviving a completed reorg takes the conservative arm: converge to the fork and re-sync the branch — data loss is resyncable, a peak over a reverted coin set is not.

Proven red-first in node/tests/reorg.rs against a real store wrapped with one armed fault (the daemon FaultStore precedent): `crash_before_branch_apply_leaves_store_untouched_and_retry_succeeds` and `crash_before_peak_flip_leaves_store_untouched_and_retry_succeeds` (sqlite) and `mmap_crash_mid_reorg_leaves_store_untouched_and_retry_survives_reopen` all failed on the streamed reorg with the exact torn invariant ("the peak's coin set is exactly the sum of its chain's deltas") and pass on the single batch; stores/tests/mmap_contract.rs fabricates the two kill windows byte-for-byte (inside the publish; after meta, before journal removal) and proves open-time convergence + a clean re-driven reorg; stores/tests/coin_store.rs and postgres_contract.rs pin dropped-batch-untouched / committed-as-one-unit on the SQL backends (the Postgres test ran against a live Postgres 16).

Consensus relevance: crash consistency only — fork choice and validation were already correct; this closes the window where a mid-reorg crash left a peak whose coin set was not the sum of its chain's deltas.

DIVERGENCE-30 — mempool timelock admission: heights off by one, seconds/relative locks never checked (found by Tier-0 audit) — FIXED

chia 2.7.1: `add_spend_bundle` (mempool_manager.py) validates EVERY time-lock at admission against the peak — "the height and time we pass in here represent the previous transaction block's height and timestamp. In the mempool, the most recent peak block we've received will be the previous transaction block, from the point-of-view of the next block to be farmed" (:787-798). Three pieces: - `compute_assert_height` (:79-124) folds each spend's `height_relative` / `seconds_relative` / `before_*_relative` into effective absolute bounds using the removals' CONFIRMED coin records (ephemeral removals get a synthesized record confirmed at peak+1 with the peak's timestamp, :721-737). - `check_time_locks` (chia.consensus, formerly Python in-tree) checks all absolute/birth/relative locks against `(peak.height, peak.timestamp)` — heights STRICTLY `assert_height <= peak.height`. - The pend-vs-fail split (:825-829): only `ASSERT_HEIGHT_ABSOLUTE_FAILED` and `ASSERT_HEIGHT_RELATIVE_FAILED` get `MempoolInclusionStatus.PENDING` (cached in `PendingTxCache`, drained at `assert_height <= peak.height`, pending_tx_cache.py:97-98); every other time-lock error — seconds absolute AND relative, all `ASSERT_BEFORE_*`, both birth asserts — is FAILED and dropped. Impossible constraints (`assert_before_height <= assert_height`, `assert_before_seconds <= assert_seconds`, :806-811) fail outright before the pending decision. `can_replace` (:1096-1189) compares the EFFECTIVE `assert_height` / `assert_before_height` / `assert_before_seconds` of

the conflicting items against the replacement's. `mempool.new_tx_block` (mempool.py:329-345) expires resident items whose `assert_before_seconds <= timestamp OR assert_before_height <= height` on every new transaction-block peak.

dg_xch before: admission checked ONLY `conds.before_height_absolute` and `conds.height_absolute` — and the height boundary was off by one (`height_absolute > peak + 1` parked, so `assert_height == peak + 1` sat RESIDENT; the pending drain used the same `<= height + 1`). Our own `validate_block_conditions` (core block_generator) fails a block when `conds.height_absolute > ctx.block_height` where `block_height` is the previous transaction block's height — so a resident `peak+1` item included in a block built on that peak is rejected by our own engine and by mainnet: an invalid-block bug the moment the producer assembles generators from the mempool. Seconds-absolute, both birth asserts, and every per-spend relative lock were never evaluated at admission; RBF timelock equality compared raw absolutes; the pool carried no peak timestamp; nothing expired resident `ASSERT_BEFORE` items on a new peak.

dg_xch now (node/src/mempool.rs): admission fetches the removals' coin records once (double-spend check + time-lock frame, same query), synthesizes peak+1/peak-timestamp records for ephemeral removals, computes `EffectiveTimelocks` (the `compute_assert_height` port), rejects impossible before<=assert combinations outright, then runs the full `check_time_locks` port against `(peak.height, peak.timestamp)` with chia's exact pend-vs-fail split — unmet height locks park keyed by EFFECTIVE assert height (drained at `assert_height <= peak.height`, pending eviction highest-assert-height-first like PendingTxCache), unmet seconds/birth locks fail (`TimelockNotMet`), passed befores fail (`Expired`). `can_replace` equality runs on the stored effective values. `new_peak` takes the transaction-block timestamp, expires resident items on chia's `new_tx_block` boundary, and the daemon only advances the mempool peak on TRANSACTION blocks (chia new_peak:884-886).

Proven red-first in node/tests/t060_mempool.rs: six tests failed on the old code with the exact wrong behaviors (peak+1 resident; unmet seconds-absolute resident; unmet height-relative resident; passed before-height-relative resident; impossible constraints parked; RBF raw-vs-effective conflict) and pass on the fix, including `assert_pool_valid_at_peak` — every resident item must validate under core's `validate_block_conditions` at the admitting peak, the invariant the off-by-one broke.

Consensus relevance: direct — a mempool-built block carrying an `assert_height == peak+1` (or unmet-seconds / unmet-relative) spend is invalid under both our consensus and mainnet's.

DIVERGENCE-31 — request_blocks serving: no range cap, headers-only flag ignored, unsolicited block replies tolerated (found by Tier-1 p2p audit) — FIXED

chia 2.7.1: `full_node_api.py` block serving: - `request_blocks` (:420-480) rejects BEFORE touching the store when `end_height < start_height` OR `end_height - start_height > constants.MAX_BLOCK_COUNT_PER_REQUESTS` (=32, `default_constants.py:77`), answering `RejectBlocks(start_height, end_height)`. The range is INCLUSIVE and the size is checked before the +1 bump — the in-source comment calls out that "MAX_BLOCK_COUNT_PER_REQUESTS is off

by one", so a conforming node serves at most **33** blocks (`end - start == 32` serves; `> 32` rejects). Any unknown height in the range (interior hole or future span, `blockchain.contains_height` false / `height_to_hash` None) also answers `RejectBlocks(start, end)` (:432-436). There is no fork/header-hash handling — `RequestBlocks` carries no header hash; heights resolve against the canonical chain only. - `include_transaction_block=false` strips ONLY `transactions_generator` (`block.replace(transactions_generator=None)`) — in `request_block` (:415-416) and per block in `request_blocks` (:438-451). `transactions_info` and `transactions_generator_ref_list` ship untouched. - Unsolicited/late `respond_block` / `respond_blocks` / `reject_block` / `reject_blocks` ban the sender: `peer.close(RATE_LIMITER_BAN_SECONDS)` (:482-517). (`respond_proof_of_weight` only logs — no ban, :398-401.)

dg_xch before (p2p/src/handlers.rs dispatch): `request_blocks` iterated `start..=end` with no width check — a hostile peer could request the whole chain into ONE `RespondBlocks` (unbounded store reads + a giant serialization: memory/CPU self-DoS); an inverted range answered an empty `RespondBlocks` instead of a reject. `include_transaction_block` was decoded and ignored in BOTH `request_block` and `request_blocks` — headers-only pulls got full generators (also a wire-visible mismatch to a strict client expecting chia's smaller stripped message). The four block replies were absent from the dispatch filter, so an unsolicited one fell to the read loop's "No Matches" error log and the connection stayed open — a free spam channel.

dg_xch now: `request_blocks` mirrors chia's checks in chia's order — inverted/oversized range rejects before the store is touched (the cap comes from `FullNodeApi::max_block_count_per_requests()`, default 32 = `ConsensusConstants::max_block_count_per_requests`; the daemon overrides it with its network constants like the existing `request_header_blocks` cap) — and both block handlers strip ONLY `transactions_generator` when `include_transaction_block=false`. The four block replies now match the dispatch filter and close the connection + evict the peer from the map: a SOLICITED reply is consumed by the read loop's correlation-id fast path (`PendingRequests::deliver` + `continue`) before the handler scan ever runs, so reaching dispatch at all means unsolicited or late — exactly the two cases chia closes on. Our own sync never trips the cap: every fetch path bounds its span to 32 blocks (`SyncConfig::batch = 32` reservations, the follow driver's `FOLLOW_BATCH = 32` band, the anchor's 32-block chunks), i.e. `end - start = 31 <= 32`.

Residual delta (documented, not built here): chia's `peer.close(ban_time)` also enters the peer in a timed ban list that refuses re-connection for `RATE_LIMITER_BAN_SECONDS`; our p2p layer has no ban list (established at T0-2), so a closed spammer can immediately reconnect. The enforceable action — immediate close + peer-map eviction — is what ships; ban plumbing is a separate, deliberate piece of work.

Proven red-first in `p2p/tests/t040_handlers.rs`: on the old code an oversized span (34 blocks) was served in full, an inverted range answered an empty `RespondBlocks`, headers-only pulls returned generators, and all four unsolicited replies left the peer connected; all pass on the fix, with the cap's off-by-one (33-block span still serves) and the untouched `transactions_info`/`generator_ref_list` locked in.

Consensus relevance: none (serving surface only) — but the missing cap was a remote resource-exhaustion vector and the strip mismatch a protocol-parity gap visible to strict peers.

DIVERGENCE-32 — inbound rate limiting advertised RateLimitsV2 but enforced a v1-subset with no max-size, no aggregate window, and a silent-drop posture (found by Tier-1 p2p audit) — FIXED

chia 2.7.1: the rate limiter is three files acting as one per-connection gate (`chia/server/rate_limit_numbers.py`, `chia/server/rate_limits.py`, enforced in `chia/server/ws_connection.py`):

- **The numbers (`rate_limit_numbers.py`).** Every message type carries an `RLSettings(aggregate_limit, frequency, max_size, max_total_size)` — a per-60s frequency, a per-MESSAGE `max_size`, and a per-60s cumulative size (defaulting to `frequency * max_size` when `None`). A handful of response types are `Unlimited(max_size)` — no frequency, only a per-message size cap, because their corresponding request already bounds them. `get_rate_limits_to_use` selects the table per connection: when BOTH peers advertise `RATE_LIMITS_V2` the v2 overlay is applied on top of v1 (`{**rate_limits[1], **rate_limits[2]}`, :49-57) — v2 raises the light-wallet/sync query frequencies (e.g. `request_puzzle_solution` 1000→5000, `request_additions` 500→50000) and moves several of them OUT of the aggregate window (`aggregate_limit` flips true→false). Otherwise v1 applies.
- **The aggregate window.** Alongside the per-type budgets, ALL `aggregate_limit=true` (non-tx) types share ONE per-connection window: 1000 messages / 60s and 100 MiB (`aggregate_limit = RLSettings(False, 1000, 0, 100*1024*1024)`, :35-39), checked BEFORE the per-type budget. "tx" types (`aggregate_limit=false`: transactions, block-headers, none_response) and `Unlimited` responses are exempt — they scale with TPS.
- **The machinery (`rate_limits.py`).** A monotonic 60s slot; on slot change every counter resets. Each message: aggregate check first (count then size), then per-type count / per-message `max_size` / per-type cumulative size. Crucially the `finally` commits the counters unconditionally for INCOMING messages (the bytes were already received, so a persistent violator keeps climbing) and only-if-allowed for OUTGOING.
- **The posture (`ws_connection.py:996-1009`).** Inbound violation on a non-localhost peer → `close(RATE_LIMITER_BAN_SECONDS)` (=300s, `protocol_timing.py:8`). The limiter runs in `_read_one_message` for EVERY inbound message BEFORE dispatch, so solicited replies count too. Outbound (`_send_message`:860-885): we throttle OURSELVES against the composed limits; a would-be violation is logged (debounced 30s) and, except `respond_peers`, re-queued after 1s (`_wait_and_retry`) — chia queues, it does not drop. `respond_blocks`/`reject_blocks` are `Unlimited` precisely so a peer can sync from another under the same limits without either side self-throttling.

dg_xch before (`p2p/src/rate_limits.rs`, `handlers.rs`): the limiter was a node-wide per-peer map keyed by `Bytes32`, implementing "chia v1, the served subset" as `Budget{frequency, max_total_size}` with **no `max_size`**, **no aggregate window**, and **no capability selection** — we advertised `RateLimitsV2` (`shared::CAPABILITIES`) but enforced v1-subset numbers, so a compliant v2 peer sending within its v2 budget (e.g. `request_puzzle_solution` above 1000/min) had messages **silently dropped** while the connection stayed open. It ran only in `FullNodeHandler::handle`, i.e. only for `served()` types that reach the handler — solicited replies consumed by the read

loop's correlation fast-path (`RespondBlocks` etc.) were **never counted** (the `RespondProofOfWeight` table entry was dead code). Violation → drop the one message, keep the connection (a documented "gentler first posture").

dg_xch now: a faithful port lives in `dg_xch_core::protocols::rate_limits` — the full v1 table, the v2 overlay, `compose`/`get_rate_limits_to_use`, per-message `max_size`, the non-tx aggregate window, and the incoming-commits-unconditionally machinery. It is a **per-connection** limiter (chia's model, one per `WSChiaConnection`), owned by the read loop (`ReadStream`), which charges EVERY inbound message against the composed limits BEFORE the correlation fast-path or handler scan — so solicited `RespondBlocks` bursts count too and the `RespondProofOfWeight` entry is live. The peer's handshake capabilities are recorded on `SocketPeer` (server role in the `Handshake` dispatch arm; outbound role in `WsClient::build` after the oneshot handshake) and drive the v1/v2 selection per connection. A violation now **closes the connection and evicts the peer** (matching chia's `close(...)`; using the T1-1 `close_peer` plumbing). Installed on the full-node paths only — the daemon's inbound listener (`WebsocketServer::rate_limited`) and the p2p outbound dialer (`WsClientConfig::rate_limited`); harvester/farmer/ wallet client roles leave it off, matching chia which only disconnects on the full-node side. The handler-level limiter was removed (enforcement is now solely at the read loop, once, like chia).

Our own sync never self-trips: `respond_blocks`/`reject_blocks` are `Unlimited` (size-only, 50 MiB / 100 B), so a rapid solicited fetch burst at our real FOLLOW/prefetch cadence stays within budget — proven at the wire in `p2p/tests/t045_rate_limits.rs::solicited_respond_blocks_burst_does_not_self_trip_the_client_limiter`.

Residual delta (documented, not built here): two pieces of chia's posture are deliberately deferred, consistent with DIVERGENCE-27/31: 1. **Timed ban list.** chia's `close(RATE_LIMITER_BAN_SECONDS)` also enters the peer in a 300s ban list; our p2p layer has no ban list (T0-2), so a closed spammer can immediately reconnect. The enforceable action — immediate close + peer-map eviction — is what ships. 2. **Outbound self-throttle.** chia re-queues would-be-oversending outbound messages after 1s. We do not yet self-throttle sends; our own outbound traffic is already bounded by the sync scheduler's 32-block reservation window and the `SEND_TIMEOUT`, and every fetch reply type is `Unlimited`, so we do not currently generate traffic that a correct peer's limiter would reject. If a future path could, the minimal correct version is a per-type outbound `RateLimiter` (the same machinery, `incoming=false`) on the send path — deferred until demonstrably needed.

Proven red-first: on the old code an oversized single message passed (`RequestBlock` at 200 B > the 100 B cap) and N distinct non-tx types each under their own budget jointly exceeded 1000/60s without tripping anything; both are violations on the fix. Unit coverage of the numbers/aggregate/max-size/v1-v2 selection is in `dg_xch_core::protocols::rate_limits::tests`; the close-and-evict posture and the solicited-fetch self-safety are in `p2p/tests/t045_rate_limits.rs`.

Consensus relevance: none (transport/serving surface only) — but the gap was a live interop defect (compliant mainnet peers throttled below their negotiated budget) and a resource-exhaustion vector (no per-message size cap, no cross-type aggregate).

DIVERGENCE-33 — wallet subscription serving: unbounded initial-state response, sub-parity subscription cap, over-requested initial query, unbounded concurrent wallet serves (found by Tier-1 RPC/wallet audit) — FIXED

chia 2.7.1: the light-wallet subscription surface is bounded four ways (`chia/full_node/full_node_api.py`, `chia/full_node/subscriptions.py`, `chia/full_node/coin_store.py`, `chia/full_node/hint_store.py`, `chia/util/limited_semaphore.py`):

1. **The initial-state response budget.** `register_for_ph_updates` (api.py:1805) reads `max_subscribe_response_items(peer)` — 100,000 untrusted / 500,000 trusted (api.py:2221-2225, initial-config.yaml:441/448) — and spends ONE budget across the whole reply: the puzzle-hash query runs with `max_items=budget` (a decremented `LIMIT` per batch INSIDE the SQL, coin_store.py:486-517), then `max_items -= len(states)`, then the hint-id lookup runs under the remainder (`hint_store.get_coin_ids_multi(..., max_items=remaining)`, itself `LIMIT`-bounded, hint_store.py:42), and the hinted states are read with `max_items=len(hint_coin_ids)`. Truncation is **silent to the wallet**: chia logs (WARNING if trusted, else INFO, api.py:1846-1861) and answers anyway, echoing the REQUESTED puzzle hashes. `register_for_coin_updates` caps its read the same way (api.py:1874/1885).
2. **The subscription cap.** `max_subscriptions(peer)` = 200,000 untrusted / 2,000,000 trusted (api.py:2215-2219, yaml:437/444), enforced as a COMBINED puzzle+coin count per peer (`peer_subscription_count`, subscriptions.py:82).
3. **Query only what was subscribed.** The ph initial-state query runs on `add_puzzle_subscriptions`' RETURN — the newly-added set, with in-request duplicates, already-subscribed hashes, and cap overflow filtered out (subscriptions.py:87-119, api.py:1816). The coin path instead SLICES the request to `max_subscriptions`, subscribes + queries the sliced list, and echoes the sliced list (api.py:1879-1889; in-request duplicates deliberately stay queryable — the api.py:1876 TODO).
4. **Concurrency.** `wallet_sync_api_sem = LimitedSemaphore.create(active_limit=2, waiting_limit=20)` (api.py:166) wraps the heavy block-delta scans of `request_additions` / `request_removals` (api.py:1397, 1461); an acquire beyond active+waiting raises immediately and the handler REJECTS (`RejectAdditionsRequest` / `RejectRemovalsRequest`, api.py:1450, 1530).

dg_xch before the fix: none of the four. (1) `register_for_ph_updates` materialized EVERY matching coin state into one `RespondToPhUpdates` — the store cap was a fixed 50k per query with the hint-id loop fully unbounded, so one dust-storm puzzle hash was a memory + wire DoS on a public listener. (2) The per-peer cap was 100,000 combined (half of chia's untrusted number). (3) The initial query ran on the RAW request list — overflow-dropped and already-subscribed hashes were still scanned. (4) Nothing bounded concurrent additions/removals scans beyond the read-loop rate limiter.

The fix (`full-node/src/daemon.rs`, `full-node/src/wallet.rs`, `stores/src/traits.rs` + the three backends):

- `max_items` is now a parameter of `CoinStore::get_coin_states_by_puzzle_hashes`, `get_coin_states_by_ids`,

and `get_coins_for_hint` — the limit lives IN the query (running `LIMIT` budget on sqlite/postgres, scan cut-off on mmap), never fetch-then-truncate. `MAX_COIN_STATES` (50,000) remains the store-tier default for callers without a budget, matching chia's parameter default.
- `StoreApi.max_subscribe_response_items` (100,000 = chia untrusted) threads chia's exact budget flow through `ph_initial_states`: ph query under the budget, decrement, hint-id lookup under the remainder, hinted states at `len(hint_ids)`, dedup by coin id, silent truncation (log-only). The coin register read runs under the same budget.
- `WalletNotifier::register_for_*` now return the newly-added set; the ph handler queries ONLY that set, the coin handler slices the request to `max_subscriptions()` and echoes the sliced list (chia's asymmetry preserved verbatim). `MAX_ITEMS_PER_SUBSCRIBER` raised to 200,000.
- `LimitedSemaphore` (full-node/src/wallet.rs) is a faithful port of chia's: active semaphore + available-count check-then-decrement, immediate `LimitedSemaphoreFull` on overflow. `Node.wallet_sync_sem` (active=2, waiting=20) is shared by the inbound server api and every outbound connection's api; `additions`/`removals` acquire it and reject on overflow.

Residual delta (documented, not built): chia's TRUSTED tier (`trusted_max_subscribe_items` = 2,000,000, `trusted_max_subscribe_response_items` = 500,000) is not implemented — this node has no trusted-peer concept anywhere (consistent with DIVERGENCE-27/31/32 posture), so every peer gets the untrusted numbers exactly.

Proven red-first on the pre-fix code: a register whose matching set exceeded an injected budget returned everything (8 vs 5; ph+hint 7 vs 5); a re-registration re-served the full initial state; an overflow-dropped hash still fed the query; the coin request was neither sliced nor budget-capped; a full wallet-sync semaphore still served additions/removals; and the cap sat at 100k vs 200k. All are enforced now: `full-node/src/daemon.rs::tests::wallet_queries` (budget, hint-budget share, repeat-registration, overflow filter, coin slice/budget, semaphore reject), `full-node/tests/wallet.rs` (cap number, newly-added-set return, LimitedSemaphore semantics), `stores/tests/coin_store.rs` (`coin_state_queries_are_bounded_by_max_items` — the limit is in the query on every backend).

Consensus relevance: none (wallet-serving surface only) — but the gap was a resource-exhaustion vector: one `RegisterForPhUpdates` on a dust-storm puzzle hash could stream an unbounded coin-state set into a single message, and unbounded concurrent additions/removals scans could pile onto the DB behind the per-message rate limiter.

DIVERGENCE-34 — restart-resume repair: cache-only consensus walks with zero window margin, floor-invisible backfilled records, hole-blind repair (found by Tier-1 sync/stores audit; the live mm + pg-b restart-resume stall) — FIXED

chia 2.7.1: none of this class can happen in chia, structurally:

1. **The consensus walks fall back to the DB.** `Blockchain.block_record` / `get_block_record_from_db` and the height-map warmup (`chia/consensus/blockchain.py`; `full_node.py:1193–1205`) mean every retarget/SES/deficit walk that misses the in-memory cache reads the store. `BLOCKS_CACHE_SIZE` (EPOCH_BLOCKS + 4·MAX_SUB_SLOT_BLOCKS = 5,120, `default_constants.py`) is a HINT, not a wall.
2. **`height_to_hash` is persisted and rebuilt wholesale**, so a restart re-sees every record the process ever stored, main-chain or not-yet-peaked.

dg_xch (pre-fix): three restart-resume holes that jointly livelocked the mm and pg-b fleet legs (sqlite genesis legs short-circuit at floor 0 and never saw it):

1. **Cache-only walks, zero margin.** The engine's stage-path walks read `self.cache.records()` with NO store fallback (`node/src/engine.rs` `derive_required_iters` / `compute_record_from_header`; a miss is `core/src/consensus/mod.rs::missing` → "block record not found"). The walk cache was a flat `BLOCK_RECORD_WINDOW = 5,120` while the deepest first-sub-epoch retarget anchor reads 5,503 records (`difficulty_record_depth`, offset 382-regime) — NEGATIVE margin at the worst alignment, and a restart warm re-warmed the same short window. A miss was unrepairable: the consumer's MissingRecord recovery re-ran `resume_repair`, which re-warmed the same window → same NotFound → livelock. (The daemon's own `difficulty_records_map` had already solved this class with the record-window + store fallback — the NODE crate's stage path never got the same treatment.)
2. **Backfilled records never became floor-visible.** `get_block_record_by_height` is main-chain-only on every backend (sqlite/ postgres `in_main_chain = 1`; mmap `heights.dat` populated only by `set_peak` — `add_block_records` inserts `main: false`). Epoch-depth backfill writes CANDIDATE records, so `resume_repair`'s by-height binary-search floor measured the confirmed span base forever → every restart of an anchored leg re-ran the weight-proof fetch + multi-minute 6-phase validation + ~4,700-block header backfill before the first follow step.
3. **Mid-span holes defeated warm and floor alike.** The warm stops at the first missing record (correct — it is a hash walk), but the by-height floor search assumed hole-free monotone presence: a hole ABOVE the floor (Postgres `synchronous_commit=off` dropping a clean candidate-batch suffix; mmap losing an un-inserted link batch) read as "nothing to repair" while every stage walk died crossing the hole. Worse (found red-first): sqlite `set_peak`'s fork walk breaks at the hole with `fork_height = -1` and demotes the ENTIRE main chain below it, so the by-height view collapses further on the next peak.

Fix (mirrors chia's model):

- `consensus_walk_window(constants)` (core difficulty_adjustment) = EPOCH + SUB_EPOCH + 6·MAX_SUB_SLOT = 5,760 on mainnet — proven ≥ every `difficulty_record_depth` + trailing slack by test. The engine walk cache and warm are sized by it; `full-node`'s `record_window_capacity` delegates to it (one source of truth).
- **Store fallback at the stage path** (chia cache-miss → DB-read parity): `Engine::prepare_delta` catches the walk's NotFound, runs `repair_walk_ancestry` (prev-hash walk, cache-then-store, depth-bounded, ascending re-insert), rewinds the VDF/sig sink to its checkpoint, and retries ONCE. Zero new work on the hot path; a genuine store gap still surfaces as MissingRecord (the driver's backfill signal) — pinned by test.
- **Floor by prev-hash walk** (`full-node/src/resume_floor.rs`): candidates count, holes are detected exactly where they break the chain (`Broken { stop }` anchors

the header backfill AT the hole), and a re-armed repair whose floor reads clean deepens the backfill one epoch below the reach point instead of replying "nothing to do" forever.

Proven red-first on the pre-fix code: the restart warm filled 5,120 of the 5,760-record walk window; a stage over a store-present record died "block record not found" (`node/tests/restart_resume.rs` kill-point classes D/E); the anchored shape measured `Broken { stop: <anchor> }` (spurious weight-proof re-validation) and the hole shape never converged even after the hole record landed (`full-node/src/resume_floor.rs` tests, sqlite AND mmap; postgres shares sqlite's by-height SQL verbatim and the same generic by-hash walk). All green post-fix.

Operational payoff: the deployed mm and pg-b legs' EXISTING stores converge on the next roll — the floor walk sees their backfilled candidates (no WP re-validation on boot), a detected hole is backfilled at the hole, and the engine reads repaired records through the store fallback even when the warm predates the repair.

Consensus relevance: none on accepted blocks (the walks compute the same values; the fallback only widens the record SOURCE, and the record content is identical — the store rows were written by the same derivation). The fix changes only whether a node can make progress after a restart.

DIVERGENCE-35 — no long-sync band for a deep MID-CHAIN gap: the far-behind decision only fired from a near-empty store (found by Tier-1 sync audit, G2) — FIXED

chia 2.7.1: `new_peak` (chia/full_node/full_node.py:840-873) is a band ladder keyed ONLY on the gap, never on the local height:

1. within `short_sync_blocks_behind_threshold` (20, initial-config.yaml:363) → `short_sync_backtrack` (block-by-block);
2. a claimed tip below `WEIGHT_PROOF_RECENT_BLOCKS` (1000, default_constants.py:72) → batch sync from zero, no weight proof;
3. within `sync_blocks_behind_threshold` (300, initial-config.yaml:360) → `short_sync_batch` from `peak - 6`;
4. **past 300 behind → `_sync()` (full_node.py:1021-1121), REGARDLESS of local height**: heaviest-peak selection, weight-proof fetch + validation, the fork point of the proof's sub-epoch summaries against the LOCAL chain (`WeightProofHandler.get_fork_point`, weight_proof.py:644-664: last positional summary agreement, backed off two sub-epochs, `<= 2` clamped to genesis), the `check_fork_next_block` peer probe (a peer's peak+1 whose prev IS our peak lifts the no-fork point to the local peak, full_node.py:3516-3526), then `sync_from_fork_point` — batch long-sync FROM THE FORK POINT, with `Blockchain.add_block`'s fork choice reorging across the gap when the fork point is below the peak.

dg_xch (pre-fix): the only far-behind gate was `wants_fast_sync = local < 1000 && gap > 1000` — the WP-anchored bulk sync fired ONLY from a near-empty store. A node at height 2M offline for weeks (gap ~50k) ground 32-block FOLLOW windows across the whole gap with **no weight-proof trust anchor for the claimed target**, and a reorg

during the offline period wedged the follow (orphan → 5-deep backtrack → DeepFork with no mid-chain answer). Proven red-first: `daemon.rs::deep_mid_chain_gap_selects_the_wp_anchored_long_sync_band` (local 2,000,000 / claimed 2,050,000 chose FOLLOW).

Fix (chia-parity band decision + fork point, wired into the existing machinery — nothing rebuilt):

- **Band:** `wants_long_sync = claimed >= 1000 && gap > 300` (`SYNC_BLOCKS_BEHIND_THRESHOLD = 300`, chia's number) at both decision seams (`sync_driver`, `follow_fill_claimed`). The near-empty sub-case (`wants_fast_sync`) keeps the unchanged from-zero recent-chain landing; genesis-sync/`--sync-from` guards unchanged.
- **Mid-chain anchor:** `Node::ensure_long_sync_anchor` — once per landing: `validated_proof` (cached WP validate), `wp_fork_point` (`node/src/sync/mod.rs`: chia get_fork_point at sub-epoch granularity, walked TOP-DOWN from our peak — summaries hash-chain via `prev_subepoch_summary_hash`, so one positional match proves the prefix; a checkpoint-anchored store has no genesis summary map to walk bottom-up), the `next_block_connects` probe (chia node_next_block_check verbatim), and the `long_sync_plan` mapping (probe-connect → extend from peak; otherwise rewind from the conservative point; Unknown → fail closed, never batch-sync blind). The producer's FOLLOW fill is gated on the anchor, so the batch download never outruns the trust anchor; once anchored, the detached fetch/confirm pipeline batch-syncs the gap and the band exits through follow → near-tip as the gap closes (target re-polls ride the T0-3 PeakBook).
- **Reorg across the gap:** `Chaser::long_sync_reland_reporting` re-follows forward windows from the fork point: identical blocks confirm AlreadyHave, a divergent branch stages as orphan candidates and flips through the engine's SINGLE-TRANSACTION reorg (T0-4) the moment it outweighs the stale peak; bounded by `LONG_SYNC_REORG_MARGIN` (96) past the stale tip, fail closed if the attested branch never outweighs.

Tests: `full-node` daemon band/plan/transition units (red-first), `node/tests/wp_fork_point.rs` (fixture summaries: agreement to sub-epoch K starts near K, never zero; divergence → Diverged at the two-below back-off; `<= 2` clamp; Unknown fail-closed), `node/tests/long_sync_reland.rs` (35-deep reorg across the gap reorgs atomically onto the heavier branch — depth metric 30 — and the agreeing-chain reland is AlreadyHave-then-extend).

Deliberate residual divergences (documented, not bugs): (1) our long sync still lands recent-chain-style ONLY for the near-empty store; the mid-chain gap is fully downloaded+validated from the fork point (chia downloads from the fork point in both cases — the from-zero recent-chain jump remains the pre-existing light-landing divergence). (2) chia's `check_fork_next_block` runs under the blockchain priority mutex; ours runs in the driver band with the same effect (the anchor is established before the producer may fill). (3) The anchor marker persists per validated tip (matching the deliberate "reuse ANY validated proof" cache); a SECOND offline period reuses the ladder's orphan → backtrack → DeepFork recovery rather than re-proving, until the cached proof is superseded.

DIVERGENCE-36 — the producer built only EMPTY blocks: no mempool→block-generator path (found by Tier-2 mempool audit #1) — FIXED

chia 2.7.1: a winning `declare_proof_of_space` farms the mempool (chia/full_node/full_node_api.py:950-975): with the `block_creation` config default `1`, the node calls `mempool_manager.create_block_generator2(tx_peak.header_hash, timeout=2.0)` → `mempool.create_block_generator2` (chia/full_node/mempool.py:708-868), which iterates the pool in fee priority (`SELECT name, fee FROM tx ORDER BY priority DESC, seq ASC`, mempool.py:722), batches spend bundles through the chia_rs `BlockBuilder` (crates/chia-consensus/src/build_compressed_block.rs — an incremental back-reference-compressing `Serializer` over the quoted spend-list tree `(q . ((parent puzzle amount solution) ...))`), enforces `MAX_SPENDS_PER_BLOCK` 6000 at build (mempool.py:72, :762), applies the skip/stop heuristics (`MAX_SKIPPED_ITEMS` 10, `MIN_COST_THRESHOLD` 6_000_000; the rust builder itself caps at 6 skipped batches), guards the fee sum, and finalizes into a `NewBlockGenerator` whose cost is the builder's true block cost. The legacy `create_block_generator` (config 0, mempool.py:516-581) selects with identical semantics, emits `solution_generator_backrefs`, and RE-RUNS the emitted program (`run_block_generator2` + `DONT_VALIDATE_SIGNATURE`, mempool.py:549-559) taking the re-run cost as the block cost. `create_unfinished_block` (chia/consensus/block_creation.py) splices program/signature/cost/ additions/removals into `TransactionsInfo`; the empty-block coercion (full_node_api.py:1104-1112) nulls `new_block_gen` when `candidate_sp_total_iters <= tx_peak.total_iters`; `mempool_manager`'s wrapper gates on `peak.header_hash == last_tb_header_hash`.

dg_xch (before): `assemble_candidate` (node/src/farmer.rs) hardcoded `transactions = None` — "dg_xch produce path builds only empty blocks today" — and the daemon's `try_build_candidate` only LOGGED the empty-block coercion. A real farm win produced a valid but fee-less empty block; the mempool's `items_by_fee()` had no consumer on the produce path.

Fix (red-first; the red run's exact failure: "GAP (Tier-2 #1): winning candidate with a fee-paying resident mempool item must carry a block generator — the produce path built an empty block"): - `Mempool::create_block_generator` (node/src/mempool.rs): fee-priority selection over `items_by_fee()` with chia's guards — fee-sum overflow, the 6000-spend cap, `MAX_SKIPPED_ITEMS` 10 (break ON the tenth skip), `MIN_COST_THRESHOLD` stop, a 2s wall-clock budget, the budget `MAX_BLOCK_COST_CLVM − BLOCK_OVERHEAD` (`QUOTE_BYTES·COST_PER_BYTE + QUOTE_EXECUTION_COST` = 24_020, chia mempool_manager.py) spent at each item's admission cost (identical accounting to chia's `conds.cost`). The emitted program is the chia_rs `solution_generator` PLAIN byte format — `solution_generator_from_coin_spends` (core/src/consensus/block_generator.rs) reverses the spend order to match chia_rs's prepend-built list, pinned byte-for-byte against chia_rs 0.42.1's own `test_solution_generator` vector (`chia_rs_solution_generator_byte_parity`). The emitted generator is then RE-RUN through our own `execute_block_generator_result` (chia's mempool.py:549-559 re-run) and the re-run cost is the declared `TransactionsInfo.cost`. Signatures aggregate via `bls_bindings::aggregate_signatures` (chia `AugSchemeMPL.aggregate`). - `assemble_candidate` takes `transactions: Option<&BlockTransactions>`; `CandidatePrev` carries `prev_transaction_block_height`. - The daemon applies the empty-block coercion for

real and gates the payload on `may_build_transactions` (tx block ∧ ¬coerced ∧ mempool frame == the candidate's prev tx block — chia's `peak.header_hash == last_tb_header_hash`, height-keyed here; every resident item is re-validated against the store on each new peak, so the height gate carries the same guarantee, failing closed to an empty block). - Resident ⇒ includable, verified: admission and the expiry sweep hold every resident item to the pool peak's `(height, timestamp)` frame — the SAME previous-transaction-block frame the assembled block validates under (t060 `assert_pool_valid_at_peak`; t080 `resident_timelocked_item_is_includable_at_the_peak_frame` proves an `assert_height == peak` resident passes `validate_transaction_block`).

Tests: `node/tests/t080_block_assembly.rs` (11 — the red-first gap test now green end-to-end through `validate_transaction_block` with SF9 live; fee priority + seq tiebreak; the 6000-spend cap; skip-break on the 10th / survive 9; MIN_COST_THRESHOLD stop; cost == item cost + BLOCK_OVERHEAD == independent re-run; two-key aggregate-signature verify; post-SF9 form ff01/canonical/empty refs; empty-mempool byte-path unit; no-peak fail closed), core `chia_rs_solution_generator_byte_parity`, full-node `mempool_payload_gate_matches_chia` + the walks test's `prev_transaction_block_height` pins. Regression: `producer_differential` (7,953 real mainnet blocks byte-identical) and `declare_proof_of_space` (6,244 real proofs) stay green — the empty-mempool path's bytes are untouched (the new path only activates with includable residents).

Deliberate residual divergences (documented, not bugs): (1) NO back-reference compression — chia 2.7.1's `BlockBuilder` emits the backref-compressed serialization of the same tree; dg_xch has no incremental backref `Serializer`, so the plain form costs more bytes and packs fewer transactions near the block-cost limit (blocks remain fully consensus-valid; chia's own config-0 path is the same tree). (2) NO dedup/fast-forward processing during assembly — the mempool tracks no dedup/FF eligibility (audit gap #7, separate item), so items assemble at FULL cost (correct, not cost-optimal) and chia's `PRIORITY_TX_THRESHOLD` dedup shortcut never engages; the batch/rollback compression retry of generator2 is likewise meaningless without a compressor. (3) NO backup-empty candidate: chia stores a second `new_block_gen=None` candidate (`add_candidate_block(..., backup=True)`, full_node_api.py:1177-1201) and, when synchronously farming the tx candidate raises, re-registers the backup and re-requests farmer signatures; dg_xch's self-farmed validation is deferred to the driver (`ub_inbox`), so the fallback needs a driver→farmer re-request seam — deferred; the producer's re-run gate closes the generator-bug half of what the backup guards against. (4) The mempool frame gate is height-keyed, not header-hash-keyed (the mempool holds no hashes); combined with per-peak store revalidation of every resident item it fails closed to an empty block on any mismatch.

DIVERGENCE-37 — `SendTransaction` (wallet p2p, code 48) was silently dropped: no dispatch arm, no `TransactionAck` (found by Tier-1 RPC/wallet audit, headline #1) — FIXED

chia 2.7.1: `full_node_api.py::send_transaction` (:1535-1569) ALWAYS answers a wallet's spend submit with `TransactionAck(txid, uint8(status), error)` where status is `MempoolInclusionStatus` SUCCESS(1)/PENDING(2)/FAILED(3) and `error`

is the `Err` member NAME as a string (`error.name`, :1560 — wallets string-match it). The exact flow: (1) fast path — bundle already in the mempool → ack SUCCESS/None immediately (:1539-1543); (2) queue the bundle on the `TransactionQueue` (trusted peers high-priority), acking FAILED/"Transaction queue full" on overflow; (3) `anyio.fail_after(45)` await on the entry's done-future — on TIMEOUT ack PENDING/None, otherwise ack the validator's `(status, error.name)`, with one more idempotence rescue: a FAILED/PENDING result whose bundle IS now resident acks SUCCESS/None (:1563-1566). `add_transaction` (full_node.py:2872-2988) supplies the statuses: not synced → (FAILED, NO_TRANSACTIONS_WHILE_SYNCING) before any CLVM (:2882-2885); resident → (SUCCESS, None); seen/in-flight → (FAILED, ALREADY_INCLUDING_TRANSACTION); pre-validate ValueError → (FAILED, INVALID_SPEND_BUNDLE), ValidationError → (FAILED, e.code); then `mempool_manager.add_spend_bundle` (:552-621) with exactly TWO pending classes — an unmet ASSERT_HEIGHT lock parks in the PendingTxCache → (PENDING, ASSERT_HEIGHT_{ABSOLUTE,RELATIVE}_FAILED) (:826-827, :613-618) and a losing replace-by-fee conflict goes to the conflict cache → (PENDING, MEMPOOL_CONFLICT) (:609-613); every other reject is FAILED. On SUCCESS only, `broadcast_added_tx` re-gossips `NewTransaction` to full-node peers ("Only broadcast successful transactions, not pending ones. Otherwise it's a DOS vector", full_node.py:2977-2980).

dg_xch before: code 48 fell through the dispatch `_ => Ok(())` — no ack, ever. Every wallet submit over p2p (the chia reference wallet AND Sage, whose submit path blocks on the TransactionAck: sage-wallet/src/utils/submit.rs:59 → wallet_peer.rs:214) hung to its timeout against us. The only working tx ingress was HTTP `push_tx`. The wallet-protocol gap doc did not even list the message.

Fix (one admission seam, chia-shaped acks): - `full-node/src/tx_admission.rs` — `admit_spend_bundle`, the ONE `add_transaction` seam now shared by all three ingress surfaces (HTTP `push_tx`, the gossip validator worker, and the p2p `SendTransaction` arm): chia-exact idempotent fast path (resident → success, no CLVM, no re-announce; residency RE-CHECKED under the same mempool lock that admits, mirroring full_node.py:2938-2940's under-lock race rescue), CLVM + aggregate-signature validation at next-block height, and the `NewTransaction` announce queued iff NEWLY resident — a p2p-admitted spend re-gossips exactly like a pushed one. - `p2p/src/handlers.rs` — the code-48 dispatch arm + `FullNodeApi::send_transaction` (store-blind default: FAILED/ NO_TRANSACTIONS_WHILE_SYNCING — a node that cannot validate must not claim success); the ack echoes the request id so wallet-side request/response correlation resolves. - `full-node/src/daemon.rs` — the `StoreApi` impl: synced gate before any CLVM, then the seam, then chia's `(status, Err.name)` mapping. - `node/src/mempool.rs` — `TimelockFailure` carried through `MempoolError::{Pending,Expired,TimelockNotMet,ImpossibleTimelock}` so the ack names the failing condition with chia's exact `Err` member name (ASSERT_HEIGHT_ABSOLUTE_FAILED, ASSERT_SECONDS_ABSOLUTE_FAILED, ASSERT_BEFORE_, *ASSERT_MY_BIRTH_*, IMPOSSIBLE_*_CONSTRAINTS), plus `MempoolError::ack()` — chia's pend-vs-fail split as data.

Tests: `full-node/tests/send_transaction.rs` (6, red-first over a real websocket link on the production handler stacks: ack SUCCESS + resident + announce queued; duplicate idempotent-SUCCESS with NO second announce; FAILED with BAD_AGGREGATE_SIGNATURE / DOUBLE_SPEND / ASSERT_SECONDS_ABSOLUTE_FAILED; unmet-height PENDING

ASSERT_HEIGHT_ABSOLUTE_FAILED, parked not resident, nothing announced; not-synced FAILED NO_TRANSACTIONS_WHILE_SYNCING; the admitted spend flows to `create_block_generator` — the T2-1 tie-in, SendTransaction → mempool → block).

Deliberate residual divergences (documented, not bugs): (1) NO trusted tier — chia queues trusted peers high-priority and sizes their limits separately; no trusted-peer concept node-wide (the DIVERGENCE-33 precedent). (2) NO TransactionQueue — admission runs synchronously on the per-message dispatch task (bounded by the read-loop rate limiter, SendTransaction 5000/min), so chia's queue-full FAILED and 45s-timeout PENDING(None) acks cannot arise; every ack carries the definitive result. (3) NO conflict cache — a losing replace-by-fee conflict acks chia's wire-exact (PENDING, MEMPOOL_CONFLICT) but is not parked for retry; the wallet's resubmit-on-new-peak covers it. (4) NO seen/in-flight cache — a concurrent duplicate submit serializes on the mempool lock and acks idempotent SUCCESS instead of chia's (FAILED, ALREADY_INCLUDING_TRANSACTION); strictly more useful to the wallet, same end state. (5) Validation rejects map through `ChiaError` (numerically chia's `Err`): the common names are exact (BAD_AGGREGATE_SIGNATURE, WRONG_PUZZLE_HASH, DOUBLE_SPEND, BLOCK_COST_EXCEEDS_MAX, INVALID_CONDITION, …); CLVM failures without a precise chia name fall back to INVALID_SPEND_BUNDLE — chia's own ValueError catch-all name (full_node.py:2907-2911).

DIVERGENCE-38 — CLVM BLS operators (opcodes 49..=59) unimplemented: `op_unknown` no-op with token cost (live `InvalidBlockCost` wedge, mainnet 9,179,161) — FIXED

Status: FIXED on `full-node` (this commit). Outcome (i) of the live-wedge triage: HEAD reproduced the deployed rejection byte-for-byte.

The live failure (2026-08-21)

`dg-xch-node-0` — the only tip-following leg — wedged at peak 9,179,160: every follow step `from:9179161 to:9179192` failed with `sync node error: consensus rejected block: InvalidBlockCost`, the peak froze (~06:00 UTC), health went 503 and the liveness probe restart-looped the pod (54 restarts at triage start, 61 by capture time). Deployed image v0.1.20260820135748 (= 4ec10f4); the divergence is present at HEAD (7f2491e) too — nothing between the deploy and HEAD touched the dialect dispatch.

The bisect (the DIV-17 method, three instruments)

The window 9,179,155..9,179,200 was wire-captured from a synced chia 2.x node (`chia-mainnet-0.iriga`, RequestBlocks/RespondBlocks with `include_transaction_block=true` — the new `block_fetch` corpus tool) and driven through `run_body_expensive`:

1. **Block-level repro:** of the 10 generator blocks in the window, exactly one fails — 9,179,161: ours 8,872,498,290 vs declared 8,878,096,231, **-5,597,941** (an UNDER-count; DIV-17 was an over-count).
2. **Both-sides decomposition** (chia-consensus 0.42.1 oracle — the exact crate chia-blockchain 2.7.x pins — `run_block_generator2` on the same generator bytes): chia computes the declared cost exactly, with byte cost (324,792,000), condition cost (153,600,000), spend set (54 spends, 62 creates, 35 agg-sigs) and signature validation all IDENTICAL to ours — the whole delta sits in pure CLVM execution cost.
3. **Per-spend + per-op diff** (clvmr 0.17.7, the clvmr chia-consensus 0.42.1 resolves): spend 33 carries the entire delta (clvmr 6,293,040 vs ours 695,099). Its per-opcode histogram: `0x37` bls_g2_negate 2,164 + `0x38` bls_map_to_g1 195,780 + `0x3a` bls_pairing_identity 5,400,000 = 5,597,944; ours charged those three calls the unknown-op token cost (3 total) — delta 5,597,941, exact.

Root cause

dg_xch path: `core/src/clvm/dialect.rs` — the dispatch table had `// 49..=59 - BLS operators, not yet implemented` and fell through to `op_unknown`: nil result, unknown-op token cost.

chia oracle: clvmr 0.17.7 `src/chia_dialect.rs:228-238` dispatches opcodes 49..=59 (`bls_g1_subtract` … `bls_verify`, `src/bls_ops.rs`) UNCONDITIONALLY in the base dialect — the 2.0 hard fork moved the BLS extension outside the `softfork` guard, and clvmr replays EVERY height with them wired. 9,179,161 is simply the first block our follow range hit whose generator executes one outside the guard (an in-puzzle pairing check: g2-negate + map-to-g1 + a 2-pair `bls_pairing_identity`).

Beyond the cost: `bls_pairing_identity`/`bls_verify` must TERMINATE the program on a failed check (clvmr `BLSPairingIdentityFailed`/ `BLSVerifyFailed`). The unknown-op no-op returned nil unconditionally — a soundness hole that would have silently accepted whatever the puzzle asserted (this block's pairing happens to verify; the cost mismatch is what made the wedge visible).

Fix

`core/src/clvm/bls_ops.rs` (new): all 11 operators ported 1:1 from clvmr 0.17.7 `src/bls_ops.rs` — cost constants verbatim, clvmr's argument traversal (silent stop on a non-pair tail for the n-ary ops, strict `get_args`/`get_varargs` elsewhere), `check_cost` placement (including map_to_g2's single post-DST check), strict point parsing per chia-bls 0.42.1 (`PublicKey`/`Signature::from_bytes` over raw blst, reusing the DIV-24-hardened `bls_g1` helpers), `mod_group_order` scalar reduction, and the AUG-scheme `aggregate_pairing`/`aggregate_verify` pairing glue (chia-bls `signature.rs` ported verbatim). Dispatch arms 49..=59 wired unconditionally (`core/src/clvm/dialect.rs`), matching clvmr's base dialect. `RELAXED_BLS` (chia_rs hard-fork-2 flag: negate accepts invalid points) added to the flag ladder at `hard_fork2_height` (`core/src/clvm/utils.rs`, `BlockGeneratorFlags::for_height`) — strict validation everywhere below it, exactly chia_rs `get_flags_for_height_and_constants`.

Error-class note: a failed pairing/verify raises `ClvmError` and the body path maps it to `InvalidBlockSolution` (this VM's generic condition-extraction reject), where chia surfaces clvmr's dedicated `BLSPairingIdentityFailed`/`BLSVerifyFailed` codes; both reject the block — name-level parity only, consistent with the op_raise precedent.

Evidence

- `node/tests/cost_wall_9179161.rs` + committed fixtures `node/tests/fixtures/blocks_9179155_9179200 (two RespondBlocks frames)`: the REAL wedge window, red at HEAD (the exact -5,597,941 at 9,179,161), green with the port — every generator block in the window must execute to EXACT declared cost with its aggregate signature verified. This window wedged production; the fixture is permanent.
- `core/tests/clvm.rs bls_ops_cost_and_value_match_clvmr`: 13 vectors — all 11 operators cost- AND value-pinned against clvmr 0.17.7 (whole-program totals), plus the raise-on-failure cases (non-identity pairing, tampered bls_verify) and the empty-verify/identity semantics.
- Both-sides decomposition after the fix: cost 8,878,096,231 == declared, execution/condition components byte-equal to chia-consensus 0.42.1.
- Full regression: workspace clippy `-D warnings`, core/node/full-node suites (t060 22, t070 13/13, t080 11 among them), era replays a–e (+ era-e-first24 SF9 boundary), and producer_differential per era (8193+8193+6145+8193+4002 = 34,726 blocks, non-tx tier byte-identical) — all green.

Consensus relevance: blocking — any mainnet block whose generator executes a BLS operator outside the softfork guard (increasingly common post-hard-fork) fails cost validation, and a failed in-puzzle pairing check would have been silently ACCEPTED. `block_fetch` (`full-node/src/bin/block_fetch.rs`) is the new peer-sourced corpus-capture sibling of `corpus-import`, kept for the next live wedge.

DIVERGENCE-39 — the modern wallet-sync surface (codes 94-103) silently dropped + no `NewPeakWallet` push: Sage could not sync (Tier-2 "Sage protocol") — FIXED

chia 2.7.1: the post-`RegisterForPhUpdates` wallet sync every modern light wallet (Sage, chia-wallet-sdk) actually uses:

- `full_node_api.py::request_puzzle_state` (:2002-2078, code 98) — the PAGED puzzle-hash history read. Truncate the request list to `CoinStore.MAX_PUZZLE_HASH_BATCH_SIZE` (= `SQLITE_MAX_VARIABLE_NUMBER - 10` = 32,690, coin_store.py:588 / db_wrapper.py:24) and dedup order-preserving (:2010-2012); reorg-consistency — `request.header_hash` must equal `height_to_hash(previous_height)` (GENESIS_CHALLENGE when `previous_height=None`), else `RejectPuzzleState(REORG)` (:2014-2023); subscription-cap check `len(hashes) + peer_subscription_count > max_subscriptions(peer)` → `RejectPuzzleState(EXCEEDED_SUBSCRIPTION_LIMIT)` when `subscribe_when_finished`, run BEFORE and AFTER the store await (:2026-2040, :2066-2070); the read is `coin_store.batch_coin_states_by_puzzle_hashes` (:590-703): plain match UNION hint match (`include_hinted` — the CAT/NFT discovery join), the `include_spent`/`include_unspent` (`spent_index>0` / `<=0`) and `min_amount` filters, `ORDER BY MAX(confirmed_index, spent_index) ASC LIMIT max_items+1` with `max_items = max_subscribe_response_items` (100k untrusted); over the cap, the `max_items+1`-th row's activity height becomes the next

page's floor and EVERY trailing row at that height is dropped — a block's states are never split across pages (:689-703); the response reports `height = next_min_height - 1` (peak when finished) + `height_to_hash(height)` (:2055-2064, no-peak/ unresolvable → REORG reject); `subscribe_when_finished` subscribes ONLY on the finished page (:2072-2074). NO sync gate — an unsynced node serves from the chain it has (there is no `synced` check in the handler).

- `request_coin_state` (:2085-2141, code 101) — the coin-id twin: truncate to `max_subscribe_response_items` + dedup (:2090-2093), same reorg check, same double cap check, `get_coin_states_by_ids(True, ids, min_height, max_items)` (:552), `subscribe` subscribes unconditionally (no is_finished concept), echoes the truncated deduped id list. Never consults the peak.
- `request_remove_puzzle_subscriptions` / `request_remove_coin_subscriptions` (:1961-1995, codes 94/96) — `None` = clear ALL returning the prior set; `Some(list)` = remove the subset returning what was actually removed (subscriptions.py:155-199).
- `NewPeakWallet` (code 50): `full_node.py on_connect` (:1000-1008) greets a WALLET-type peer with the current peak (`fork_point = the peak height` on connect); `update_wallets` (:1535-1571) broadcasts it to ALL wallet peers on every peak advance (after the per-subscriber `CoinStateUpdate` deltas), `fork_point = wallet_update.fork_height`.
- `MempoolItemsAdded/Removed` (104/105) ride behind `Capability.MEMPOOL_UPDATES` (:2160-2161, :2197-2198).

dg_xch before: codes 94/96/98/101 fell through the dispatch `_ => Ok(())` — silently dropped, every request timed out. And `NewPeakWallet` was never sent AT ALL (zero references in p2p/full-node). Sage drops any peer that has not produced a `NewPeakWallet` within 2s of connecting (sage-wallet sync_manager/peer_discovery.rs `try_add_peer`, options.rs `initial_peak = 2s`), so the wallet surface was DOUBLY unreachable: Sage disconnected before asking, and had it asked, its sync loop (wallet_sync.rs: `RequestCoinState` subscribe + paged `RequestPuzzleState` until `is_finished`) would have hung on the first request.

Fix: - `stores/src/traits.rs` — `CoinStore::batch_coin_states_by_puzzle_hashes` (coin-index tier): chia's paged read, `(Vec<CoinState>, Option<u32>)` with the whole-height page cut; shared bounded-merge + page-cut helpers (each per-hash probe is ordered + `LIMIT max_items+1` IN the query, the merge keeps only the smallest `max_items+1` by (height, coin id) — T1-3 idiom, never fetch-then-truncate); `MAX_PUZZLE_HASH_BATCH_SIZE = 32_690`. Implemented on all three backends (sqlite/postgres per-hash indexed probes with `MAX()`/`GREATEST()` ordering + the `coin_hint` sub-select join + big-endian-blob `amount >= ?`; mmap as a scan with the hint-log sweep) + `Arc` forwarding. - `p2p/src/handlers.rs` — dispatch arms for 94/96/98/101 (`served()` + labels updated), `PuzzleStateReply`/`CoinStateReply` (Respond carries the first-registration delivery receiver, bridged by the SAME `spawn_coin_state_forwarder` as the register path), and the on-connect greeting: after answering a WALLET-type handshake, `api.wallet_peak()` → `NewPeakWallet` (fork_point = peak height). - `full-node/src/daemon.rs` — the `StoreApi` impls (chia-ordered checks as above; `height_to_hash` = main-chain record by height); the Node now OWNS the inbound `PeerMap` and `notify_new_peak` broadcasts `NewPeakWallet` to every wallet-type inbound peer after the `CoinStateUpdate` deltas (wallet-peer snapshot first — no store read and no per-block cost with zero wallets connected). - `full-node/src/wallet.rs` — `WalletNotifier::{peer_subscription_count, remove_ph_subscriptions, remove_coin_subscriptions}` (reverse-index scrubbed; the peer's delivery channel survives removal for re-subscribe).

Tests (red-first): `full-node/tests/puzzle_state.rs` — 8 wire tests on the production stacks, all RED on the pre-fix code (requests timed out; no NewPeakWallet): answered 98/101/94/96; REORG rejects (wrong hash at a height, wrong genesis anchor) + matching-previous-peak serves; unsynced node still serves (chia has no sync gate here); on-connect NewPeakWallet within Sage's 2s budget with chia-exact fields; peak-advance broadcast through the REAL follow path (wallet-type peer receives, full-node-type does not); and the ACCEPTANCE — a message-for-message replay of Sage's sync sequence (connect → NewPeakWallet → RequestCoinState subscribe → paged RequestPuzzleState with threaded (height, header_hash) until is_finished → live CoinStateUpdate push → remove-all unsubscribe returning the subscribed sets → no further pushes) converging to a seeded state that includes a hinted CAT-shaped coin. Api-seam tests (daemon.rs wallet_queries, injectable budgets): the Sage page loop to convergence at budget 4 (no height split, no duplicates, header_hash chain passes the reorg check each round), the shared 94-103 subscription cap (single-request, cumulative, cross-leg ph+coin, remove-all frees it), coin-id budget truncation echo. Store contract tests (sqlite coin_store.rs, mmap_contract.rs, postgres_contract.rs env-gated): paging recovers everything exactly once, spent/unspent partition, min_amount (numeric on the BE blob), hint join + plain/hinted dedup, both-false short-circuit.

Deliberate residual divergences (documented, not bugs): (1) `fork_point_with_previous_peak` on the peak-advance broadcast is `height - 1` (the existing CoinStateUpdate simplification — our confirm path emits per-block deltas, so height-1 IS the fork point for every non-reorg advance; chia threads the reorg's true fork height). (2) CoinStateUpdate (per-peer forwarder task) and NewPeakWallet (inline broadcast) are not mutually ordered; chia sends deltas then the peak in one task. Sage treats both as independent events. (3) List truncation happens after decode instead of chia's `list_limits` parse-time cap (api_protocol.py:84-89 → streamable.py `parse_list_limited`) — byte-stream- and semantics-equivalent. (4) A store error mid-read rejects REORG (always-answer) where chia raises and the wallet times out. (5) MEMPOOL_UPDATES initial pushes (chia :2074/:2137) are skipped: `Capability.MEMPOOL_UPDATES` is not advertised, and chia gates those sends on the peer capability — codes 104/105 remain out of scope. (6) No trusted tier (the DIVERGENCE-33 precedent): every peer gets the untrusted 100k/200k budgets.

DIVERGENCE-40 — wallet correctness residue: no additions/removals Merkle proofs (G2), empty served `transactions_filter` (G3), hint-blind live CoinStateUpdate (Tier-2 wallet-correctness #3) — FIXED

chia 2.7.1:

- **Proofs (G2).** `full_node_api.py::request_additions` (:1372-1452) / `request_removals` (:1455-1532): a request with specific `puzzle_hashes`/`coin_names` is answered with per-item INCLUSION/EXCLUSION proofs from `chia_rs.MerkleSet` (pin `>=0.42.1, <0.43`) — additions over the `[puzzle_hash, hash_coin_ids(coin names)]` leaf pairs (:1424-1431, `hash_coin_ids` = single→`sha256(id)`, multi→sort-DESC+concat+`sha256`, coin.py:16-26), removals over the removal names with the set root ASSERTED equal to the foliage `removals_root` (:1511-1515). Per requested hash: `is_included_already_hashed` → additions proof triple `(ph, proof,`

`Some(coin_ids_proof))` when present / `(ph, proof, None)` when absent (:1433-1447); removals pair `(name, proof)` with `(name, Some(coin))` / `(name, None)` (:1516-1526). The EMPTY-`puzzle_hashes` short-circuit answers `coins=[], proofs=[]` — Some-empty, NOT None (:1392-1394); removals treats `Some([])` exactly like `None` (all removals, proofs None, :1505). Proof wire format (chia_rs `merkle_tree.rs`): node-type-prefixed pre-order — `EMPTY=0`, `TERMINAL=1`+leaf, `MIDDLE=2`, `TRUNCATED=3`+subtree hash — every level down to the leaf pair present (`pad_middles_for_proof_gen` re-expands what the hash computation collapses; `MidDbl` propagation decides the collapse points). A wallet verifies the bytes against the block header's foliage roots.

- **Filter (G3).** Served headers carry a chiabip158 filter: `request_block_header` re-runs the generator then `get_block_header(block, (removal_names, tx_additions))` — filter over every addition's puzzle hash (tx additions + `reward_claims_ incorporated`) then every removal name (generator_tools.py:26-35); `request_header_blocks`/`request_block_headers` build the same set from the COIN STORE (`get_coins_added_at_height` minus coinbase + reward claims re-added, `get_coins_removed_at_height` names, full_node_api.py:1644-1652/:1693-1700). Non-tx blocks and `return_filter=False` serve the encoded-empty `b"\x00"` (full_block_utils.py:311/:320).
- **Hints (live push).** `update_wallets` (full_node.py:1541-1546) matches each pushed coin against ph-subscriptions by coin id, its OWN puzzle hash, AND `wallet_update.hints[coin_id]` — the map built from THIS peak's create-coin hints (`ppp_result.hints`, :2101-2103, 32-byte-only). That is how a wallet subscribed to a CAT/DID/NFT outer puzzle hash sees the inner-puzzle coin land live.

dg_xch before: (G2) the specific-`puzzle_hashes`/`coin_names` arms `rejected` unconditionally — untrusted-mode legacy wallet sync was locked out; the empty-short-circuit answered `proofs=None` where chia sends `Some([])`. (G3) every served header carried a ZERO-LENGTH `transactions_filter` (not even the encoded-empty `b"\x00"`) — a silent data-miss for filter-syncing wallets; `return_filter` was accepted and ignored. (Hints) `WalletNotifier::on_new_peak` matched only `puzzle_hash` + `coin_id` — a live push for a coin hinted to a subscribed puzzle hash was silently missed (the initial-state and RequestPuzzleState reads DID hint-join, so the gap was exactly the push).

Fix:

- `core/src/consensus/merkle_set.rs` (new) — `MerkleSet` (`from_leafs`/`from_proof`/`get_root`/`generate_proof`) + `validate_merkle_proof`, ported verbatim from chia_rs `chia-consensus` 0.42.1 `merkle_tree.rs`+`merkle_set.rs` (0.37.0 and 0.42.1 are byte-identical — the encoding is consensus-frozen). Malformed proofs error (never panic): bounds-checked reads, depth cap 256, leaf-position audit, trailing-byte rejection. `hash_coin_ids` made `pub` (block_generator.rs).
- `full-node/src/daemon.rs` — `additions`: insertion-ordered ph→coins map, chia's post-read reorg re-check (:1402), `Some([])`→`proofs= Some([])`, proof path with the CNI leaf pairs + per-hash triples (chia's asserts land as rejects — never a panic, never unprovable data); `removals`: `Some([])` = trusted-all, proof path over the removal names with the root-vs-foliage-`removals_root` check (chia's assert → reject + warn), reorg re-check (:1485); `served_header_block` (coin-index): real filter from the added/removed-at-height indexes (added rows = tx additions + reward claims, exactly the rule-12 builder's element set) via the T0-1 `chia_block_filter`; `block_headers` honors `return_filter`.

- `node/src/engine.rs` — `header_block_from_full_block` default filter is now the encoded-empty `b"\x00"` (chia's no-filter representation), not a zero-length string.
- `full-node/src/wallet.rs` — `on_new_peak` takes the block's `(hint, coin_id)` pairs (the engine's `BlockDelta::hints`, already 32-byte-filtered) and `match_peers` joins the hint as a third puzzle-hash probe; the daemon threads `d.hints` through `notify_new_peak`.

Tests (red-first — each red against the pre-fix behavior with the test plumbing in place): `core/src/consensus/merkle_set.rs` unit corpus (the ported chia_rs root cases + proof round-trips incl./excl., the PINNED complete-proof hex vector, malicious/truncated/mispositioned decoder cases, cross-check vs the producer-side `merkle_set_root`); `core/tests/merkle_set_proofs.rs` — proof BYTES byte-equal to real `chia_rs` 0.42.1 output over a 10-case corpus (fixture `merkle_set_proofs_chia_rs_0_42_1.txt`, oracle-generated); daemon `wallet_queries` — additions/removals proofs SERVE and verify via `validate_merkle_proof` against block 5,000,000's REAL foliage roots, `Some([])` nuances both directions, served filter sha256 == the block's real `filter_hash` + byte-equal to the rule-12 builder + identical across all three header handlers + `return_filter=false`/non-tx → `b"\x00"`, and the served-proof-bytes-vs-chia_rs-0.42.1 gate (`merkle_proofs_5000000.json`, whose oracle asserted its roots equal the block's foliage roots at generation time); `full-node/tests/wallet.rs` — hinted create+same-block-spend push received, no-hint-re-declaration spend NOT pushed (chia's this-peak-only hint map).

Deliberate residual divergences (documented, not bugs): (1) `request_block_header` builds its filter from the coin index, not a generator re-run — content-identical for a canonical-chain block (the stored added-at-height rows ARE tx additions + reward claims; chia itself serves the range handlers from the store). (2) The non-coin-index tier serves the encoded-empty `b"\x00"` filter (it has no added/removed-at-height indexes and serves no other wallet-sync surface — the DIVERGENCE-33/39 tier precedent). (3) chia's in-handler `assert`s (root mismatch, inclusion disagreement) are rejects here — a corrupt read must not crash the daemon or serve unprovable data. (4) Like chia, the live-push hint join covers only coins hinted IN the pushed block; a later spend of a previously-hinted coin matches by coin-id subscription only (Sage subscribes coin ids of known coins).